

Microelectronics laboratory notes

Giacomo Simonetto

Second semester 2025-26

Abstract

Microelectronics laboratory notes for the second semester of the academic year 2025-26 at the University of Padua.

Contents

1	Prototyping techniques	3
1.1	PCB and PCA	3
1.2	Resistors	4
1.3	Capacitors	5
1.4	Integrated circuits	5
1.5	Breadboards	5
1.6	Software for prototyping circuits	5
2	Laboratory instrumentation	6
2.1	Oscilloscope	6
2.2	Function generator	8
2.3	Multimeter	8
2.4	Power supply	8
3	MOSFET and BJT	9
3.1	MOSFET	9
3.2	BJT	10
4	OP AMPS - performance and limitations	11
4.1	Introduction	11
4.2	Ideal op amp common circuit configurations	11
4.3	Ideal assumptions	12
4.4	Non idealities of op amps	12
5	Linear voltage regulators	15
5.1	Performance indicators	15
5.2	Simple Zener regulator	15
5.3	Transistor series voltage regulator	16
5.4	Adjustable series voltage regulator with feedback	17
5.5	Short circuit protection	17
6	Signal generators	18
6.1	Bistable circuits - Schmitt trigger	18
6.2	Triangular wave generator - Astable multivibrator	21
6.3	Sine wave generator	22
7	Output stages	23
7.1	Introduction	23
7.2	Class A - Emitter follower	23
7.3	Class B - Push-pull stage	26
7.4	Class AB	27
8	Arduino	29
8.1	Introductions to Arduino and microcontrollers	29
8.2	Programming on Arduino - pin modes, interrupts and timers	30
8.3	Simple circuits with Arduino	32
8.4	Asynchronous Serial Protocol	34
8.5	Serial Peripheral Interface - SPI	35
8.6	Libraries in Arduino	35
8.7	I ² C Protocol	37
8.8	Adafruit HX8357 LCD TFT display	40
8.9	Heart rate monitor	42

1 Prototyping techniques

1.1 PCB and PCA

Difference between PCB and PCA

- **PCB - Printed Circuit Board:** only the surface with conductive pads for mounting electronic components, along with tracks or traces, called metal lines, to connect the components, and “via holes” to connect different layers of the PCB or to mount components with through-hole technology
- **PCA - Printed Circuit Assembly:** a PCB with electronic components mounted on it

PCB classification

- classification based on **substrate materials**:
 - **FR-4:** organic composite rigid material made on fiberglass cloth with an epoxy resin binder, it is rigid, flame resistant and supports up to 135°C temperatures
 - **ceramic:** rigid inorganic ceramic substrate (Al_2O_3 , AlNi, ...) for better performance at high frequencies and thermal conductivity
 - **metal core:** metal core under the substrate (copper or aluminum) for better heat dissipation
 - **flexible substrates:** flexible materials for applications where the PCB can be bent or folded
- classification based on **physical nature**:
 - **rigid:** made of solid materials that do not bend
 - **flexible:** made of materials that can bend without breaking (for wearable devices, ...)
 - **rigid-flex:** combination of rigid and flexible sections in the same PCB, usually flex parts are used for interconnections between rigid sections (FFC - flat flexible cables)
- classification based on **number of layers**:
 - **single-sided:** only one layer of conductive tracks, components mounted on the same side
 - **double-sided:** two layers of conductive tracks, components can be mounted on both sides
 - **multi-layer:** more than two layers of conductive tracks, components can be mounted on both sides, used for complex circuits
- classification based on **production methods of the conductive pattern**:
 - **discrete conductor pattern:** conductive tracks are printed by depositing conductive material directly on the substrate without using a lithographic imaging process
 - **graphic conductor pattern:** tracks are printed using a photographic process (deposit, photoresist, UV exposure, etching) to obtain finer and more precise patterns

Component classification

- classification based on **mounting technology**:
 - **TH - Through-Hole:** components with leads or pins that go through holes in the PCB
 - **SMT - Surface-Mount:** components that are soldered directly on the surface of the PCB
- classification based on **package types**:
 - **DIP - Dual Inline Package:** TH package with two parallel pin rows, with 100 mil pitch
 - **SOP - Small Outline Package:** SMT package with pins at 25 mil pitch
 - **SSOP - Shrink SOP:** SMT package smaller than SOP
 - **TSSOP - Thin SSOP:** SMT package smaller than SSOP
 - **QFN / DFN - Dual or Quad Flat No-leads:** SMT package soldered with pads or balls

Pitch is the distance between two adjacent pins or leads of a component. It is measured in mils:

$$1 \text{ mil} = \frac{1}{1000} \text{ inch} = 0.0254 \text{ mm}$$

SMT assembly process

1. **setup**: preparation of the PCB and components for assembly
2. **screen print**: application of solder paste on the PCB pads using a stencil
3. **SMT placement**: placement of components on the PCB using a pick-and-place machine
4. **reflow**: heating the PCB to melt the solder paste and create permanent connections between components and PCB pads
5. **hand assembly**: manual placement of components that cannot be placed by the machine (large through-hole components, ...)
6. **wave solder**: soldering of through-hole components by passing the PCB over a wave of molten solder
7. **final assembly**: manual assembly of remaining components and final checks
8. **wash**: cleaning of the PCB to remove flux residues and contaminants
9. **circuit test**: testing components and connections to ensure proper functionality
10. **stress test**: testing the PCB under extreme conditions (temperature, humidity, ...)
11. **functional test**: testing the PCB in real operating conditions to verify that it performs as expected
12. **pack/ship**: packaging and shipping of the assembled PCB to customers or distributors

1.2 Resistors

Resistor color code

Resistor values are indicated with color bands on the resistor body. The first two bands on the left represent digits, the third band is a multiplier, the fourth band indicates the tolerance of the resistor value. The color code is as follows:

Color	1st Band	2nd Band	3rd Band Multiplier	4th Band Tolerance
 Black	0	0	$\times 1 \Omega$	
 Brown	1	1	$\times 10 \Omega$	$\pm 1\%$
 Red	2	2	$\times 100 \Omega$	$\pm 2\%$
 Orange	3	3	$\times 1 \text{ k}\Omega$	
 Yellow	4	4	$\times 10 \text{ k}\Omega$	
 Green	5	5	$\times 100 \text{ k}\Omega$	$\pm 0.5\%$
 Blue	6	6	$\times 1 \text{ M}\Omega$	$\pm 0.25\%$
 Violet	7	7	$\times 10 \text{ M}\Omega$	$\pm 0.10\%$
 Grey	8	8	$\times 100 \text{ M}\Omega$	$\pm 0.05\%$
 White	9	9	$\times 1 \text{ G}\Omega$	
 Gold			$\times 0.1 \Omega$	$\pm 5\%$
 Silver			$\times 0.01 \Omega$	$\pm 10\%$

E-series

Resistor values are standardized in the E-series, where the number n after “E” indicates the number of values per decades distributed logarithmically: step size = $10^{1/n}$. The most common series are:

- **E6**: 6 values per decade, 20% tolerance, 47% step size, 2 digit bands
- **E12**: 12 values per decade, 10% tolerance, 21% step size, 2 digit bands
- **E24**: 24 values per decade, 5% tolerance, 10% step size, 2 digit bands
- **E48**: 48 values per decade, 2% tolerance, 5% step size, 3 digit bands
- **E96**: 96 values per decade, 1% tolerance, 2.5% step size, 3 digit bands
- **E192**: 192 values per decade, 0.5% / 0.25% / 0.10% tolerance, 1.23% step size, 3 digit bands

In the laboratory we use E12 resistors, the most common and widely available. If the required value is not an E12 standard one, we can combine resistors in series or parallel to obtain the desired value.

1.3 Capacitors

Ceramic capacitors

Ceramic capacitors are made with one or more layers of ceramic dielectric material coated by silver plates. They are small, cheap, with low capacitance values and not polarized. They are used for decoupling and filtering DC and AC signals. Capacitance values are indicated in pF with a three-digit code:

- the first two digits represent the significant figures of the capacitance value
- the third digit indicates the number of zeros to add to the values

For example, a capacitor with code “103” has a capacitance of $10 \times 10^3 pF = 10 nF$.

Film capacitors

Film capacitors are made with a thin plastic film (polyester, polystyrene, polypropylene, polycarbonate, metalized paper, Teflon, ...) as dielectric material that separates the two electrode layers made of metal foils (zinc or aluminum) sometimes applied directly to the dielectric film and everything is rolled up to occupy a small volume. By having a wide surface area, they can achieve higher capacitance values from $5 pF$ to $100 \mu F$. They have low tolerance, good reliability and high temperature ratings. They come in different shapes and sizes, normally in rectangular forms.

Electrolytic capacitors

Electrolytic capacitors are made of a metal oxide film (aluminum) as dielectric material, covered by an electrolyte solution on one side as a second dielectric layer. Both layers are coupled with two aluminum foils as electrodes and sealed in a cylindrical aluminum case. They can achieve very high capacitance values and are used for low frequency, power supplies and decoupling applications. They are polarized, with the positive terminal connected to the aluminum oxide layer. If they are connected in reverse, the oxide layer can break down and cause the capacitor to fail.

1.4 Integrated circuits

Integrated circuits (ICs) are electronic components that contain multiple transistors, diodes, resistors and capacitors inside the same package. In the laboratory we use ICs in DIP packages, which are easy to handle and mount on breadboards. Every IC has a notch or a dot to indicate the position of pin 1 and to orient the component correctly.

1.5 Breadboards

Breadboards are prototyping tools that allow to build and test electronic circuits without soldering. They are made of two horizontal rails or buses usually used for power supply connections, and a grid of vertical connected holes for mounting components and making connections.

1.6 Software for prototyping circuits

Common prototyping software

- **Fritzing**: design circuits on a virtual breadboard and simulate their behavior
- **LTspice**: simulate electronic circuits and analyze their behavior
- **Fusion PCB**: prototype and design PCBs

Circuit simulation

Circuit study consist in solving first order equations and neglecting noise, temperature, parasitic capacitance, channel length modulation, ... Computer aided simulations are useful to observe the behavior of circuits considering also second/third order effects and non-idealities by solving mathematical models of the components provided by the manufacturer. Circuit study and design is still required to understand the behavior of the circuit and understand simulation results.

2 Laboratory instrumentation

2.1 Oscilloscope

Introduction

An oscilloscope is an electronic instrument that displays electrical signals graphically as a voltage versus time plot. Some common use cases of oscilloscopes include:

- **wave shape**: observing the shape of electrical signals
- **amplitude**: measuring the voltage levels of signals
- **frequency**: measuring the frequency of periodic signals
- **time between events**: measuring the time interval between two events in a signal
- **AC/DC components**: analyzing the DC and AC components of signals
- **noise**: detecting and analyzing noise in electrical signals

Oscilloscope architecture

- **input coupling**: switch that add a capacitor in series to the input signal to block the DC component and only let the AC component pass through, it is often used to isolate the AC signal from the DC offset and to analyze the AC characteristics of the signal
- **amplitude control**: amplifier/attenuator that adjusts the amplitude of the input signal to provide a suitable voltage range for the analog-to-digital converter (ADC)
- **channel isolation**: optical (with photodiodes) or galvanic (with transformers) isolation to isolate the input signal from the internal circuitry of the oscilloscope and to prevent interference and damage to the device
- **Analog to Digital Converter (ADC)**: samples the analog input signal at regular intervals and converts it into digital values for processing and display in real time
- **memory**: stores the digital values of the signal for processing and display
- **processor**: manage the sample storage in memory, processes the digital values of the signal to perform various functions such as triggering, measurement, graphics processing, ... and controls the user interface
- **triggering**: digital elaboration of the input signal to detect specific events (rising edge, falling edge, edge delay, pulse width, n-cycles, ...) and align the signal on the screen based on the detected events

Oscilloscope performances

- **bandwidth**: the frequency range of the oscilloscope, measured in MHz; the upper limit is the frequency at which a sinusoidal signal is attenuated by 3 dB (about 70.7% of the original amplitude); it is recommended to use an oscilloscope with a bandwidth at least 5 times higher than the frequency of the signal to be measured to ensure accurate reproduction of the signal
- **sampling rate**: the number of samples taken per second to reconstruct the input signal; as for bandwidth, it is recommended to have a sampling rate at least 5 times higher than the frequency of the signal to be measured
- **record length**: the number of samples that can be stored in memory for analysis and display; it determines the maximum time interval that can be observed on the oscilloscope screen at maximum sampling rate, obtained by the formula: $acquired\ time = record\ length / sampling\ rate$; if the time window is bigger than the acquired time, the oscilloscope will automatically reduce the sampling rate to fit the signal in the time window, which can result in a loss of detail and accuracy in the signal representation

NOTE: the oscilloscope acquisition is not continuous, it has a dead time between acquisitions during which the oscilloscope is busy to flush the memory and prepare for the next acquisition.

Oscilloscope control panel

- **channel inputs:** BNC connectors (coaxial cables) for connecting probes to the oscilloscope
- **volts/div:** adjust the vertical scale of the signal on the screen, it determines how many volts are represented by each vertical grid division
- **vertical position:** adjust the vertical position of the signal on the screen, it allows to move the signal up or down to align it with the grid or to compare it with other signals
- **time/div:** adjust the horizontal scale of the signal on the screen, it determines how much time is represented by each horizontal grid division
- **horizontal position:** adjust the horizontal position of the signal on the screen, it allows to move the signal left or right to align it with the grid or to compare it with other signals
- **trigger level:** adjust the voltage level at which the oscilloscope triggers and starts the acquisition of the signal
- other controls for trigger options, coupling, autoscale, cursors, math functions, ...

Oscilloscope display

The oscilloscope display is divided into a grid of horizontal and vertical divisions, which are used to measure the amplitude and time of the signal. The vertical divisions represent voltage levels, while the horizontal divisions represent time intervals. There are also indicators for:

- **waveform markers:** horizontal line that indicates the 0 V level on the screen for every channel
- **waveform position:** vertical line that indicates the zero time point on the screen (where the trigger event occurs)
- **cursors:** vertical and horizontal lines that can be moved by the user to measure voltage levels and time intervals
- **div settings:** indicators for the current volts/div and time/div settings for each channel
- other indicators for general settings, measurements, information, ...

Triggering

Triggering is a feature of the oscilloscope that allows to horizontally align the signal on the screen by positioning the zero time point at a specific event in the signal. The trigger can be set to detect different types of trigger events, such as:

- **rising edge:** when the voltage level becomes greater than a specified threshold
- **falling edge:** when the voltage level becomes smaller than a specified threshold
- **edge delay:** after a specified time delay from the rising or falling edge
- **pulse width:** when the signal pulse width is greater or smaller than a specified value
- **n-cycles:** after a specified number of cycles of the signal
- **digital patterns:** recognize typical patterns of digital protocols (I2C, SPI, CAN, USB, ...)

NOTE: it is not possible to trigger two signals at the same time, if two signals have different frequencies, only one of them will be triggered and correctly displayed on the screen.

Cursors

Cursors are vertical and horizontal lines that can be moved by the user to measure voltage levels and time intervals on the oscilloscope display. They can be used to measure amplitude, frequency, period, rise time, fall time, duty cycle, ... of the signal. It is a more accurate measurement method compared to counting the number of divisions on the grid.

Acquisition mode

There are different acquisition modes that determine how the oscilloscope samples and processes the input signal. Particularly important is the **average mode**, which takes multiple acquisitions of the signal and displays an average of them to filter high frequency noise and improve the signal quality. Note that the average mode slows down the acquisition process.

AC/DC coupling

The AC/DC coupling switch allows to add a capacitor in series to the input signal to filter out the DC component. This is useful to analyze the AC characteristics of the signal without being affected by the DC offset. It is important to note that when the signal frequency is too low, the capacitor starts to filter out low frequencies (10-20 Hz) that are part of the AC signal, causing voltage “droop” or “sag” in the waveform. In general it is recommended to use AC coupling for signals with frequencies over 50-200 Hz.

Input impedance and amplification/attenuation stage

In normal working conditions, the input impedance of the oscilloscope is set to HiZ (high impedance) with a value of $1\text{ M}\Omega$. Some oscilloscopes can also have a low impedance setting (LoZ) with a value of $50\ \Omega$ that bypasses the conditioning stage and directly connects the input signal to the ADC. This setting is useful for measuring high frequency signals but it can cause damage to the oscilloscope if the input voltage is bigger than 5 V.

2.2 Function generator

Introduction

A function generator is an electronic instrument that generates electrical waveforms of adjustable shapes, frequencies and amplitudes. It acts as an AC voltage source in series with a DC offset voltage source and an internal resistance.

Output impedance

The output impedance of a function generator is $50\ \Omega$ and the instrument expects to be connected to a load with the same impedance. For this reason the real output voltage is doubled to ensure the desired voltage across the load. When the function generator is connected to a load with a different impedance, the load impedance must be set accordingly. In particular, when it is connected to a HiZ load, the instrument must be set to HiZ mode.

2.3 Multimeter

The multimeter is an electronic instrument that can measure voltage, current and resistance. It is used to measure a wide voltage/current range with high accuracy.

2.4 Power supply

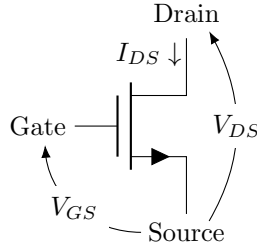
The power supply is an electronic instrument that provides a stable and adjustable DC voltage to power electronic circuits. Sometimes it is useful to link the GND terminal to the COM terminal (both of the power supply) to create a common reference point for the circuit.

3 MOSFET and BJT

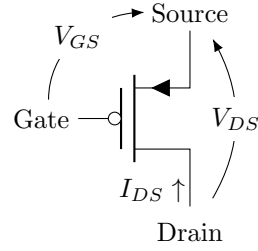
3.1 MOSFET

Scheme

A MOSFET (Metal-Oxide-Semiconductor Field-Effect Transistor) is a three-terminal semiconductor device that can be used as a switch. It has three terminals: Gate (G), Drain (D) and Source (S). The current flowing from Drain to Source (I_{DS}) is controlled by the voltage applied to the Gate (V_{GS}).



NMOS transistor



PMOS transistor

Operation regions of a NMOS transistor

- **cutoff region:** $V_{GS} < V_{TN}$, the MOSFET is off and $I_{DS} = 0$
- **linear region:** $V_{GS} > V_{TN}$ and $V_{DS} < V_{GS} - V_{TN}$, the MOSFET is on and behaves like a gate-voltage controlled resistor, I_{DS} increases linearly with V_{DS} :

$$I_{DS} = k_n V_{DS} (V_{GS} - V_{TN} - V_{DS}/2) \quad k_n = \frac{\mu_n \varepsilon}{T_{OX}} \frac{W}{L}$$

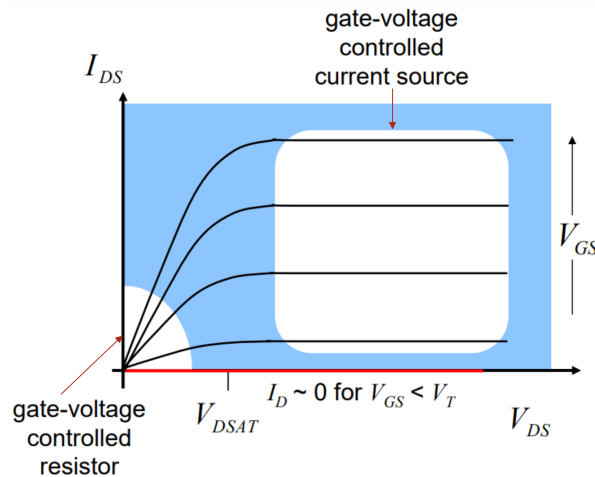
- **saturation region:** $V_{GS} > V_{TN}$ and $V_{DS} > V_{GS} - V_{TN}$, the MOSFET is on and behaves like a gate-voltage controlled current source, I_{DS} is independent from V_{DS} and increases quadratically with V_{GS} :

$$I_{DS} = \frac{k_n}{2} (V_{GS} - V_{TN})^2 \quad k_n = \frac{\mu_n \varepsilon}{T_{OX}} \frac{W}{L}$$

There are some non-idealities for short channel length transistors that affect the behavior of the MOSFET:

- **pinch-off saturation region - channel length modulation:** in saturation region, the effective channel length decreases as V_{DS} increases and the current I_{DS} increases slightly with V_{DS} instead of being constant by a factor λ (channel length modulation parameter):

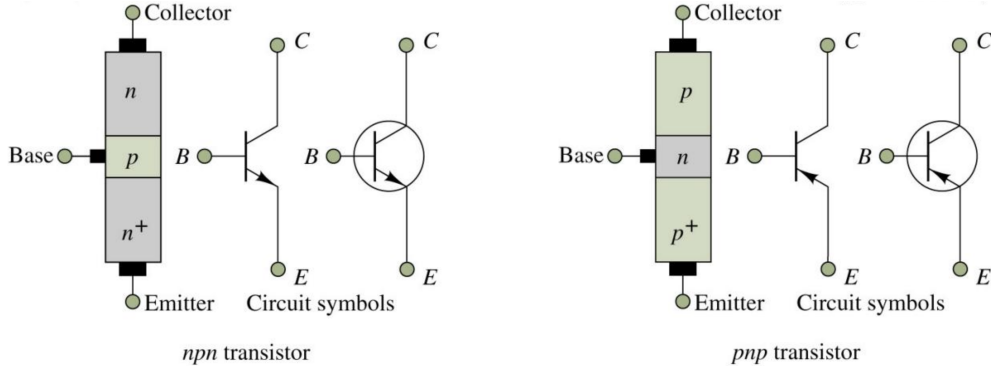
$$I_{DS} = \frac{k_n}{2} (V_{GS} - V_{TN})^2 (1 + \lambda V_{DS})$$



3.2 BJT

Scheme

A BJT (Bipolar Junction Transistor) is a three-terminal semiconductor device that can be used as a switch or as a current amplifier. It is made of three layers of semiconductor material: the emitter (E), the base (B) and the collector (C). The current flowing from collector to emitter (I_{CE}) is controlled by the current flowing into the base (I_B).



Open Collector Curve

When the base-emitter junction is forward biased and the collector is not connected to any voltage source (open collector), the BJT behaves like a diode between the base and the emitter, with a voltage drop of about 0.7 V for silicon BJTs. There is an exponential dependence between the base current I_B and the collector-emitter voltage V_{CE} (the same characteristic of a diode).

$$I_B = I_S \left(\exp \left(\frac{V_{BE}}{\eta V_T} \right) - 1 \right)$$

Operation regions of a NPN BJT

- **forward active region** - $V_C > V_B > V_E$:

the BJT is a current amplifier, the collector current I_C is proportional to the base current I_B by a factor β and the emitter current I_E is the sum of the collector and base currents; the collector-base junction is reverse biased and the base-emitter junction is forward biased:

$$I_C = \beta I_B \quad I_E = I_C + I_B = (\beta + 1)I_B \quad I_B = I_S \left(\exp \left(\frac{V_{BE}}{\eta V_T} \right) - 1 \right)$$

- **saturation region** - $V_C < V_B$ and $V_E < V_B$:

the BJT acts as a closed switch between the collector and emitter and both the collector-base and base-emitter junctions are forward biased; the collector current I_C depends only on the external circuit and previous equations are not valid anymore; the voltage drops V_{BE} and V_{CE} are approximately constant:

$$V_{BE} \approx 0.7 \text{ V} \quad V_{CE} \approx V_{CE,sat} \approx 0.2 \text{ V}$$

4 OP AMPS - performance and limitations

4.1 Introduction

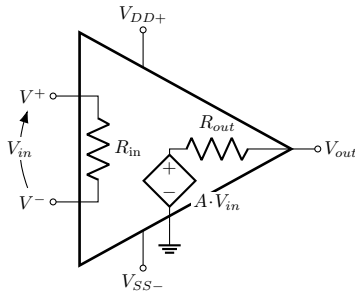
OP amps (Operational Amplifiers) are high-gain voltage amplifiers with differential inputs and a single-ended output. In ideal conditions, an OP amp has infinite gain, infinite input impedance, zero output impedance and infinite bandwidth, but in real conditions those properties are not achievable.

History

- **1953:** K2-W - first commercially available OP amp based on vacuum tubes
- **1947:** first transistor (germanium) at Bell Labs
- **1954:** first silicon transistor at Texas Instruments
- **1958:** first integrated circuit at Texas Instruments
- **1960s:** solid state op-amps
- **1968:** Fairchild invented the $\mu A741$ (commercially available as LM741), still used today

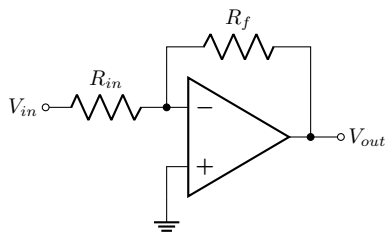
4.2 Ideal op amp common circuit configurations

Circuit model of an ideal op amp



- A : open-loop gain of the op amp
- R_{in} : input resistance of the op amp
- R_{out} : output resistance of the op amp
- V_{in} : differential input voltage ($V_{in} = V^+ - V^-$)
- V_{out} : output voltage
- V_{DD+} and V_{SS-} : power supply voltages for the op amp

Inverting amplifier



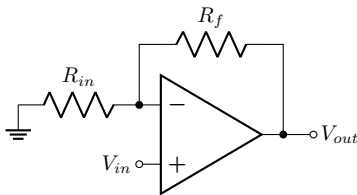
Differential input voltage $V_{in} = V^- - V^+$ must be zero to have a finite output voltage. For this reason, $V^- = V^+ = 0V$ and the inverting input is called a virtual ground potential node because it is at ground potential but not directly connected to ground.

The resistor series R_{in} and R_f acts as a voltage divider and the output voltage V_{out} is proportional to the input voltage V_{in} by a factor $-R_f/R_{in}$:

$$V_{out} = -\frac{R_f}{R_{in}} V_{in} \quad H(s) = \frac{V_{out}}{V_{in}} = -\frac{R_f}{R_{in}}$$

The output signal is out of phase (inverted) with the input signal.

Non-inverting amplifier



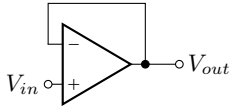
To have a finite output voltage and a zero input differential voltage, $V^+ = V^-$.

The resistor series R_{in} and R_f acts as a voltage divider and the output voltage V_{out} is proportional to the input voltage V_{in} by a factor $1 + R_f/R_{in}$:

$$V_{out} = \left(1 + \frac{R_f}{R_{in}}\right) V_{in} \quad H(s) = \frac{V_{out}}{V_{in}} = 1 + \frac{R_f}{R_{in}}$$

The output signal is in phase (non-inverted) with the input signal.

Voltage follower - buffer



The voltage follower is a specific configuration of the non-inverting amplifier with no input and feedback resistors. The output signal is ideally an exact copy of the input signal from the non inverting input.

4.3 Ideal assumptions

- **infinite open-loop gain** $A \rightarrow \infty$: the output voltage V_{out} can be arbitrarily large for any non-zero input voltage V_{in}
- **infinite input resistance** $R_{in} \rightarrow \infty$: no current flows into the input terminals (zero input-bias current $I_B = 0$) and consequently the difference between the two input-bias currents is zero (zero input-offset current $I_{OS} = 0$)
- **zero input offset voltage**: the output voltage is zero when the input voltage is zero
- **infinite common-mode rejection ratio (CMRR)**: the output voltage is only affected by the difference between the input voltages $V^+ - V^-$ and not by the common-mode voltage $(V^+ + V^-)/2$
- **zero output resistance** $R_{out} \rightarrow 0$: the output voltage V_{out} is not affected by the load connected
- **infinite output voltage range**: the output voltage V_{out} is not limited by the power supply voltages and the op amp does not saturate
- **power supply rejection**: the output voltage is not affected by power supply voltage variations
- **infinite output current**: the op amp can provide any amount of current to the load
- **infinite open-loop bandwidth**: any frequency can be amplified without attenuation
- **infinite slew rate**: the output voltage can change instantaneously in response to changes in the input differential voltage

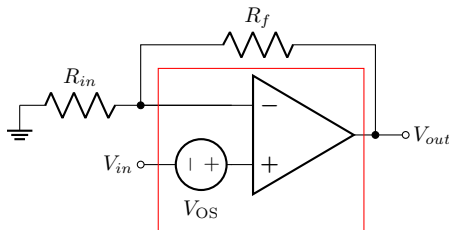
4.4 Non idealities of op amps

Input offset voltage

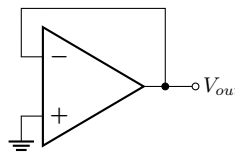
When both inputs are connected to ground, the output has a small offset voltage V_0 that keeps the output at some DC voltage level instead of 0 V. This output voltage can be measured by connecting the op amp as a voltage follower with the non inverting input connected to ground.

It is possible to create a model that include this effect by adding an ideal voltage generator in series to one of the two inputs to generates a fixed input-voltage offset V_{OS} . The V_{OS} is the equivalent differential voltage that must be provided to an ideal op amp to obtain the output voltage offset V_0 .

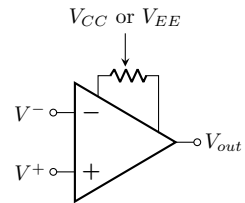
Some ICs have specific pins to add a potentiometer that can be used to compensate the input offset voltage.



Ideal op amp with zero output offset voltage V_0 with non-ideal input offset voltage V_{OS} correction highlighted in the red rectangle.



Voltage follower configuration in which the output voltage is equal to the input offset voltage V_{OS} .



Potentiometer for input offset voltage compensation

Numerical example: if a non-inverting amplifier with $R_f = 99 \text{ k}\Omega$ and $R_{in} = 1.2 \text{ k}\Omega$ has an input offset voltage of $|V_{OS}| \leq 3 \text{ mV}$, the output offset voltage V_0 is:

$$|V_0| = |V_{OS}| \cdot \left(1 + \frac{R_f}{R_{in}}\right) = 3 \text{ mV} \cdot \left(1 + \frac{99 \text{ k}\Omega}{1.2 \text{ k}\Omega}\right) = 0.25 \text{ V} \quad \longrightarrow \quad -0.25 \text{ V} < V_0 < 0.25 \text{ V}$$

Input bias and offset currents

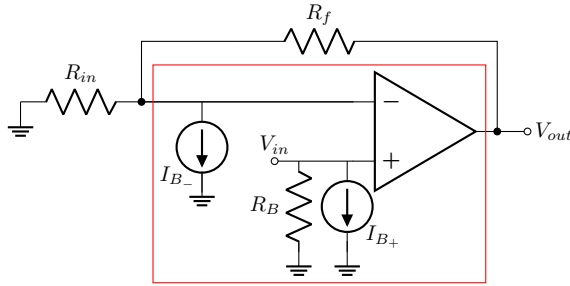
Due to finite input resistance, there are small input bias currents I_{B+} and I_{B-} that flow into the input terminals of the op amp. This happens because the input terminals are connected to the base or gate of internal transistors (BJT or MOSFET) that have a non-infinite resistance. The difference between the two input bias currents is called input offset current I_{OS} .

It is possible to create a model that include this effect by adding two ideal current generators in parallel to the input terminals to simulate the input bias currents I_{B+} and I_{B-} .

These currents cause voltage drops across the resistors R_{in} and R_f and consequently generate a small input voltage offsets that will be amplified to a considerable output voltage offset. To mitigate this effect, it is recommended to add a compensation resistor R_B between the non-inverting input and ground with a value equal to the parallel of R_{in} and R_f . Using this value, the output voltage offset depends only by the input offset current I_{OS} that is usually 5-10 times smaller than the individual input bias currents I_{B+} and I_{B-} .

$$V_{out} = V_{out,1} + V_{out,2} = I_{B-} \cdot R_f - I_{B+} \cdot R_B \cdot \left(1 + \frac{R_f}{R_{in}}\right) \quad \text{using superposition}$$

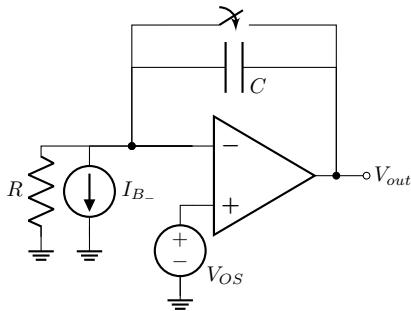
$$\text{using } R_B = R_f \parallel R_{in} = \frac{R_{in} \cdot R_f}{R_{in} + R_f} \rightarrow V_{out} = -(I_{B+} - I_{B-}) \cdot R_f = -I_{OS} \cdot R_f$$



Op amp model with current generators representing input bias currents I_{B+} and I_{B-} and compensation resistor R_B highlighted in the red rectangle.

Input bias currents in integrators

An integrator is a circuit configuration that integrates an input signal by storing charge on a capacitor C in the feedback loop. The output voltage is proportional to the integral of the input signal. The capacitor has a normally open switch in parallel to discharge the capacitor and reset the output voltage to the initial value.



NOTES:

- V_{OS} models the input offset voltage of real op amps
- I_{B-} models the input bias current of real op amps
- useful formulas: $V_C = Q_C/C$, $Q_C = I_C \cdot t$

Analyzing the circuit behavior in the two possible states of the switch:

1. when the switch is closed, op amp acts as a voltage follower and $V_{out} = V_{OS}$
2. when the switch is open, the output voltage V_{out} is:

$$V_{out}(t) = V_{start} + V_{out,1} + V_{out,2} = V_{OS} + \frac{V_{OS}}{RC}t + \frac{I_B}{C}t$$

- the input offset voltage V_{OS} that generates a current $I_{OS} = V_{OS}/R$ through the resistor R that charge the capacitor C with a voltage contribution $V_{out,1} = t \cdot V_{OS}/RC$
- the input bias current I_B charge the capacitor C with a voltage contribution $V_{out,2} = t \cdot I_B/C$
- output voltage increases linearly with time and the slope is $V_{OS}/RC + I_B/C$

Output voltage swing

The output voltage swing is the range of output voltage that an op amp can provide without saturating. It is limited by 1-2 volts below the power supply span. The saturation voltages L_+ and L_- are asymmetrical and depend on the load connected to the output and on the output current.

Output current limits

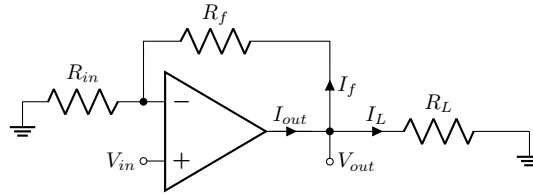
The output current of an op amp is limited by short-circuit and power dissipation protection inside the op amp.

Sometimes datasheets specify only the output swing based on the load resistance R_L , but not the maximum output current $I_{out,max}$. That value can be estimated by the following formula:

$$I_{out,max} = \frac{V_{out,max}}{R_L}$$

The output current is given by the sum of the load current I_L and the feedback current I_f . Usually R_f is much larger than R_L and the feedback current I_f is negligible compared to the load current I_L . In general, for an inverting amplifier the total output current I_{out} is:

$$I_{out} = I_L + I_f = \frac{V_{out}}{R_L} + \frac{V_{out}}{R_f + R_{in}} = V_{out}/R_{eq} \quad \text{with} \quad R_{eq} = R_L \parallel (R_f + R_{in})$$



To correctly design the circuit and choose the correct feedback resistor R_f and input resistor R_{in} of an inverting amplifier based on the load resistance R_L , the following steps can be followed:

1. calculate the minimum equivalent resistance $R_{eq} = R_L \parallel R_f$ based on $V_{out,max}$ and $I_{out,max}$
2. fix the load resistance based on the application requirements and calculate the minimum feedback resistor R_f (it is recommended to choose R_f slightly bigger than the minimum value)
3. choose the input resistor R_{in} based on the desired gain of the amplifier and the value of R_f chosen in the previous step
4. calculate the maximum output current I_{out} for the chosen resistor values and verify the requirements

It is recommended to choose resistor values between input impedance ($1\text{ M}\Omega - 10\text{ M}\Omega$) and output impedance ($10\text{ }\Omega - 100\text{ }\Omega$) of the op amp. Usually, $1\text{ k}\Omega - 100\text{ k}\Omega$ should be a good choice.

Slew rate and full-power bandwidth

Slew rate is the maximum velocity with which the output voltage can change in response to a step input voltage. It is measured in volts per microsecond ($\text{V}/\mu\text{s}$), usually in range $0.1\text{ V}/\mu\text{s} - 10\text{ V}/\mu\text{s}$. If the input slew rate is higher than the op amp slew rate, the output will be clipped, filtered or distorted.

The full-power bandwidth is the highest frequency at which a full-scale signal can be developed at the output without distortion. It is related to the slew rate SR and the maximum output voltage $V_{out,max}$ by the following formula:

$$f_{FPBW} = \frac{SR}{2\pi V_{out,max}}$$

$$\text{sinusoidal output signal: } V_{out} = V_M \sin(\omega t) \quad \rightarrow \quad \text{signal slew rate: } \left. \frac{dV_{out}}{dt} \right|_{max} = V_M \omega \cos(\omega t)|_{max} = V_M \omega$$

$$\text{signal SR should be less than the op amp SR: } V_M \omega < SR \quad \rightarrow \quad V_M < \frac{SR}{\omega}$$

$$\text{obtaining the full-power bandwidth: } \omega = 2\pi f_M < \frac{SR}{V_M} \quad \rightarrow \quad f_M < \frac{SR}{2\pi V_M}$$

5 Linear voltage regulators

5.1 Performance indicators

Line regulation

Line regulation is the measure of how well a power supply maintains a constant output voltage V_{out} for a change in the input voltage V_{in} . It is usually expressed in percentage as the ratio of the change in output voltage to the change in input voltage:

$$\text{Line regulation} = \frac{\Delta V_{out}}{\Delta V_{in}} \cdot 100\%$$

Load regulation

Load regulation is the measure of how well a power supply maintains a constant output voltage V_{out} for a change in the load conditions. It is usually expressed in percentage as the ratio between the maximum change in output voltage to the nominal output voltage:

$$\text{Load regulation} = \frac{V_{out,min \text{ load}} - V_{out,max \text{ load}}}{V_{out,nominal \text{ load}}} \cdot 100\%$$

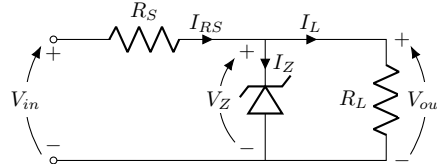
5.2 Simple Zener regulator

Zener diode characteristic

A Zener diode is a special type of diode that can operate in breakdown region (or Zener region) without being damaged. In reverse breakdown region, the Zener diode maintains a constant voltage V_Z across its terminals for a wide range of current I_Z between $I_{Z,min}$ and $I_{Z,max}$.

Simple Zener regulator

The main idea of the simple Zener regulator is to use a Zener diode in reverse bias to maintain a constant output voltage $V_{out} = V_Z$ across the load. The input voltage V_{in} is connected to the series combination of a resistor R and the Zener diode, and the load is connected in parallel to the Zener diode.



To ensure proper operation, component values must be chosen to satisfy the following conditions:

- the series resistor R_S must be large enough to limit the current through the Zener diode $I_Z < I_{Z,max}$ when load is disconnected and all the current flows through the Zener diode:

$$R_S \geq \frac{V_{in} - V_Z}{I_{Z,max}}$$

- the load resistor R_L must be large enough to ensure the minimum Zener diode current $I_Z > I_{Z,min}$; if the load is too small, current will flow only through the load and the Zener diode will not be able to maintain a constant output voltage:

$$R_L \geq \frac{V_Z}{I_{RS} - I_{Z,min}}$$

Limitations of the simple Zener regulator

- Due to constant voltage V_{RS} and constant current I_{RS} , there is a constant and significant power dissipation in the series resistor R_S even when the load is disconnected, especially when the load can require a large current I_L .
- Output voltage V_{out} changes linearly with variation of the load current I_L due to the not null Zener impedance R_Z : $V_{out} = V_Z + I_Z R_Z$, this circuit has poor load regulation.

5.3 Transistor series voltage regulator

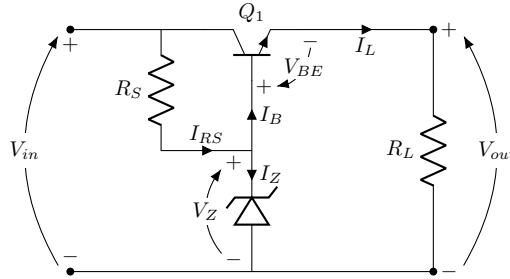
Circuit

The main idea is to use a transistor Q_1 as a variable resistor in series with the load, and to use a Zener diode to maintain a constant voltage V_Z at the base of the transistor. The output voltage V_{out} is constant and equal to the difference between the Zener voltage V_Z (constant) and the base-emitter voltage V_{BE} (also approximately constant):

$$V_{out} = V_Z - V_{BE}$$

The series resistor R_S is used to guarantee that the current through the Zener diode respects the datasheet specifications $I_{Z,max} > I_Z > I_{Z,min}$:

$$\frac{V_{in} - V_Z}{I_{Z,min} + I_B} > R_S > \frac{V_{in} - V_Z}{I_{Z,max} + I_B} \quad \text{where } I_B = \frac{I_L}{\beta} = I_S \cdot \left(e^{\frac{V_{BE}}{\eta V_T}} - 1 \right)$$



Advantages compared to the simple Zener regulator

- the circuit is more resilient to variations in the load current I_L because when the transistor works in forward active region, a change in the emitter current $I_L = I_E$ produces mainly a change in the collector current I_C and only a small change in the base current $I_B = I_C/\beta$, so the Zener diode current I_Z remains almost constant
- there is also a feedback effect: if the V_L drops, V_{BE} increases, the transistor Q_1 becomes more conductive and V_{out} increases again; vice versa if V_{out} increases
- the power dissipation in the series resistor R_S is reduced because the current through it doesn't have to be large enough to power the load I_L but only to guarantee the minimum Zener current I_Z

Problems and limitations

- with circuit heating, V_Z and V_{BE} can change in different ways, so the output voltage V_{out} is subject to temperature variations
- the output voltage V_{out} can't be controlled or adjusted to a specific value, it is fixed by the Zener voltage V_Z and the base-emitter voltage V_{BE}

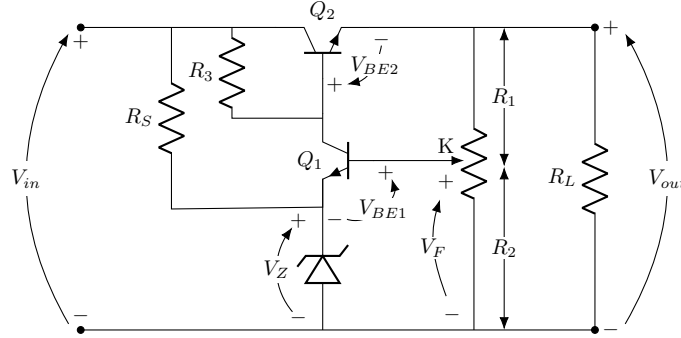
5.4 Adjustable series voltage regulator with feedback

Circuit

The idea of the circuit is to use the transistor series voltage regulator (described in the previous section) to impose a constant voltage V_F to the partial resistor R_2 of the potentiometer that acts as a voltage divider made of two resistors R_1 and R_2 . The output voltage V_{out} is given by:

$$V_{out} = I_{R2} \cdot (R_1 + R_2) = \frac{V_F}{R_2} \cdot (R_1 + R_2) = \frac{V_Z + V_{BE1}}{R_2} (R_1 + R_2)$$

The transistor Q_2 is used as a pass transistor to provide the required current to the load.



Advantages compared to the transistor series voltage regulator

- the output voltage V_{out} can be adjusted by changing the ratio between the two half resistors R_1 and R_2 of the potentiometer in the voltage divider
- the feedback effect is more complex and stronger: if V_{out} increases, V_F increases, V_{BE1} increases, I_{C1} increases, V_{R3} increases, V_{BE2} decreases, V_{BE2} decreases, I_{E2} decreases, V_{out} decreases again; vice versa if V_{out} decreases

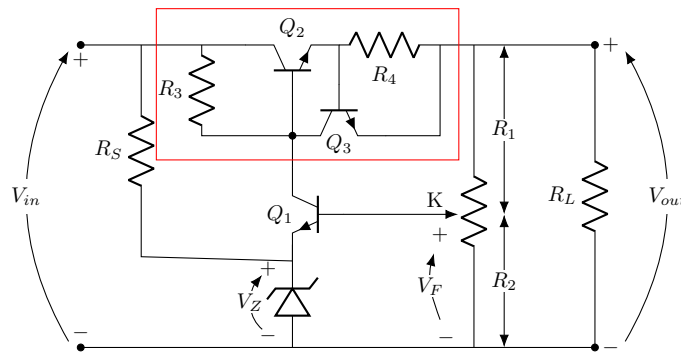
5.5 Short circuit protection

Circuit

Short circuit protection is implemented by adding a transistor Q_3 and a small resistor R_4 ($\approx 1 \Omega$) in the output branch (highlighted by the red rectangle in the circuit). When the load current I_L is too high, the voltage drop across R_4 becomes large enough to turn on the transistor Q_3 , which starts to absorb current from the resistor R_3 that produce a voltage drop V_{R3} that reduces the base voltage of Q_2 and consequently the voltage V_{BE2} that turns off Q_2 and reduces the current I_L to a safe value.

The limit current $I_{L,max}$ that can be delivered to the load is approximately given by:

$$I_{L,max} \approx \frac{V_{BE3,turn\ on}}{R_4}$$



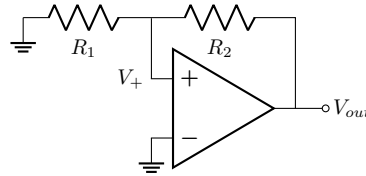
6 Signal generators

6.1 Bistable circuits - Schmitt trigger

Generic bistable circuit - Schmitt trigger

A generic bistable circuit, called also Schmitt trigger is a circuit that has two stable states and can remain in either state indefinitely until an external trigger is applied to change its state.

It can be made of an op amp with positive feedback (no virtual ground). When the circuit turns on, V_{out} , V_+ and V_- are initially at 0 V. A slight variation in the input voltage V_+ will cause the output to saturate at either L_+ or L_- depending on the sign of the variation. These two final states are the two stable states of the circuit. Due to the positive feedback, the circuit will remain in the stable state until an external trigger is applied to one of the input terminals.



Inverting bistable circuit

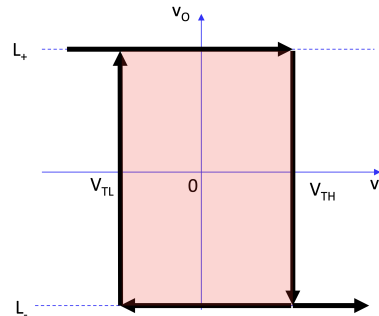
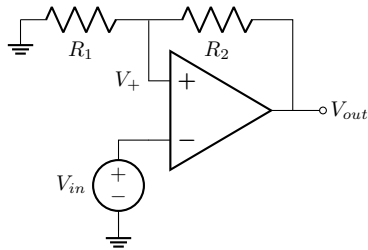
An inverting bistable circuit is a Schmitt trigger with a voltage generator (that acts as a trigger) connected to the inverting input of the op amp. It is called “inverting” because the output voltage is inverted with respect to the input trigger voltage.

The output voltage V_{out} will be at L_+ and $V_+ = L_+ \cdot R_1 / (R_1 + R_2)$ until the input trigger voltage V_{in} exceeds the positive threshold voltage $V_{TH} = L_+ \cdot R_1 / (R_1 + R_2)$. In this case the differential input voltage becomes negative ($V_+ - V_{in} < 0$), the output voltage V_{out} will switch to L_- and $V_+ = L_- \cdot R_1 / (R_1 + R_2)$.

To switch back to L_+ , the input trigger voltage V_{in} must be lower than the negative threshold voltage $V_{TL} = L_- \cdot R_1 / (R_1 + R_2)$. In this case the differential input voltage become positive again ($V_+ - V_{in} > 0$) to switch the output back to L_+ .

$$V_{out} = L_+ \iff V_+ - V_{in} = L_+ \cdot \frac{R_1}{R_1 + R_2} - V_{in} > 0 \implies V_{in} < V_{TH} = L_+ \cdot \frac{R_1}{R_1 + R_2}$$

$$V_{out} = L_- \iff V_+ - V_{in} = L_- \cdot \frac{R_1}{R_1 + R_2} - V_{in} < 0 \implies V_{in} > V_{TL} = L_- \cdot \frac{R_1}{R_1 + R_2}$$



Non inverting bistable circuit

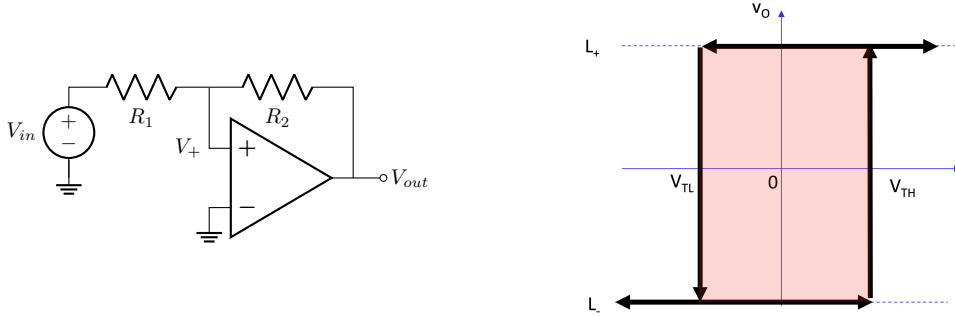
A non inverting bistable circuit is a Schmitt trigger with a voltage generator (that acts as a trigger) connected at the end of the voltage divider that feeds the non inverting input of the op amp. It is called “non inverting” because the output voltage is not inverted with respect to the input trigger voltage.

The output voltage V_{out} will be at L_+ and $V_+ = L_+ \cdot R_1/(R_1 + R_2) + V_{in} \cdot R_2/(R_1 + R_2)$ until the input trigger voltage V_{in} exceeds the negative threshold voltage $V_{TL} = -L_+ \cdot R_1/R_2$. In this case the non-inverting input voltage V_+ becomes negative, the output voltage V_{out} will switch to L_- and $V_+ = L_- \cdot R_1/(R_1 + R_2) + V_{in} \cdot R_2/(R_1 + R_2)$.

To switch back to L_+ , the input trigger voltage V_{in} must be bigger than the positive threshold voltage $V_{TH} = -L_- \cdot R_1/R_2$. In this case the non inverting input voltage V_+ become positive again to switch the output back to L_+ .

$$V_{out} = L_+ \iff V_+ = L_+ \cdot \frac{R_1}{R_1 + R_2} + V_{in} \cdot \frac{R_2}{R_1 + R_2} > 0 \implies V_{in} > V_{TL} = -L_+ \cdot \frac{R_1}{R_2}$$

$$V_{out} = L_- \iff V_+ = L_- \cdot \frac{R_1}{R_1 + R_2} + V_{in} \cdot \frac{R_2}{R_1 + R_2} < 0 \implies V_{in} < V_{TH} = -L_- \cdot \frac{R_1}{R_2}$$



Bistable circuit as a comparator

The comparator is a circuit that compares two voltages and outputs a digital signal (high or low) depending on which voltage is higher. Comparators can have hysteresis cycles (to mitigate the noise in input signal) and can be implemented with a bistable circuit. They are usually used to:

- detect signal levels with respect to a reference voltage
- convert an analog signal into a digital signal (analog to digital converter)
- implement a zero crossing detector (when the input signal crosses zero, the output changes state)

Inverting bistable circuit with offset

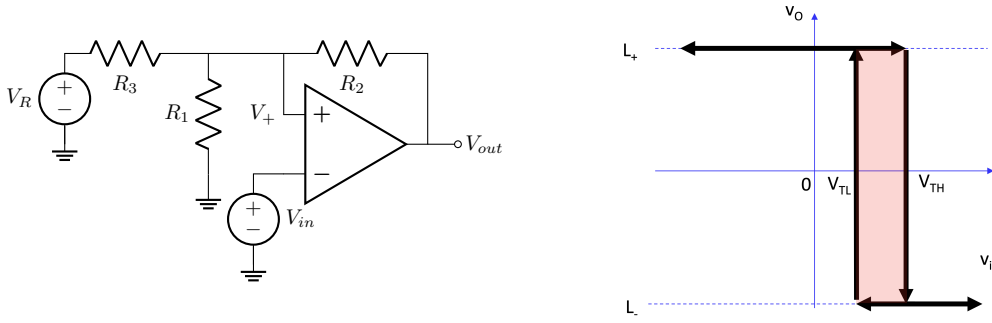
An inverting bistable circuit with offset is an inverting bistable circuit with threshold voltages that are shifted by a certain defined offset voltage. This can be achieved by adding a second voltage generator to the non inverting input of the op amp and a new resistor between the new voltage generator and the non inverting input.

The V_+ non inverting input voltage is given by the superposition of new voltage generator and the op amp output voltage:

$$V_+ = V_R \cdot \frac{R_1 \parallel R_2}{R_1 \parallel R_2 + R_3} + V_{out} \cdot \frac{R_1 \parallel R_3}{R_1 \parallel R_3 + R_2}$$

Using the same reasoning as for the inverting bistable circuit, we can derive the new threshold voltages:

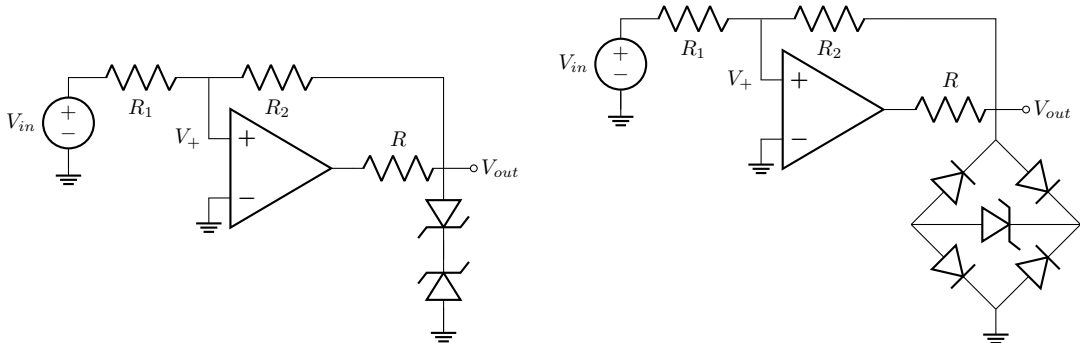
$$\begin{aligned} V_{out} = L_+ &\iff V_+ - V_{in} = V_R \cdot \frac{R_1 \parallel R_2}{R_1 \parallel R_2 + R_3} + L_+ \cdot \frac{R_1 \parallel R_3}{R_1 \parallel R_3 + R_2} - V_{in} > 0 \\ &\implies V_{in} < V_{TH} = V_R \cdot \frac{R_1 \parallel R_2}{R_1 \parallel R_2 + R_3} + L_+ \cdot \frac{R_1 \parallel R_3}{R_1 \parallel R_3 + R_2} \\ V_{out} = L_- &\iff V_+ - V_{in} = V_R \cdot \frac{R_1 \parallel R_2}{R_1 \parallel R_2 + R_3} + L_- \cdot \frac{R_1 \parallel R_3}{R_1 \parallel R_3 + R_2} - V_{in} < 0 \\ &\implies V_{in} > V_{TL} = V_R \cdot \frac{R_1 \parallel R_2}{R_1 \parallel R_2 + R_3} + L_- \cdot \frac{R_1 \parallel R_3}{R_1 \parallel R_3 + R_2} \end{aligned}$$



Fixed output voltage

Saturation output voltages L_+ and L_- of the op amp are not always defined and depend on the specific op amp used. To fix the output voltage to a defined value, we can add two zener diodes face-to-face between output and ground. There is also a resistor to bias the zener diodes and control the current through them.

Alternatively, we can use a 4-diode rectifier bridge to keep the zener diode in the correct orientation regardless of the output voltage polarity.



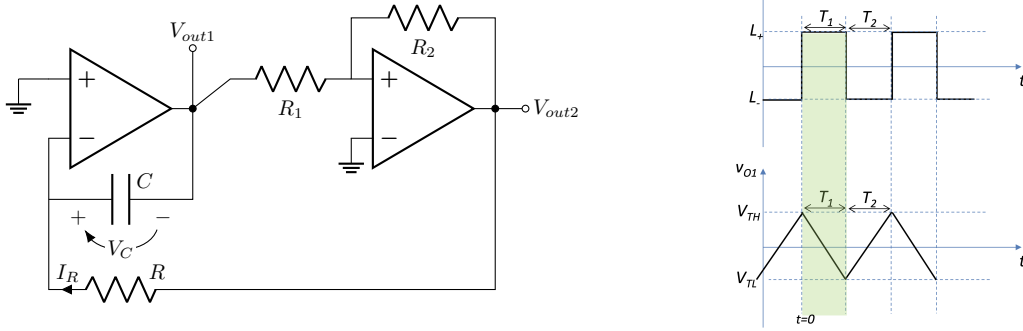
Bistable circuit with fixed output voltage using two zener diodes

Bistable circuit with fixed output voltage using a diode bridge and a zener diode

6.2 Triangular wave generator - Astable multivibrator

Circuit

Bistable circuits require an external trigger to switch between the two stable states. If we want to generate a periodic signal, we can create a loop made of a bistable circuit and an integrator. The output of the bistable circuit is a square wave that feeds the input of the integrator, which produces a triangular wave that feeds back the bistable circuit to switch its state. This new auto-oscillating circuit is called astable multivibrator. It's important to use a non inverting bistable circuit with an inverting integrator or vice versa to ensure oscillation.



Circuit behavior

- When V_{out2} is at L_+ , there is a constant current $I_R = L_+/R$ through the resistor R that charge the capacitor C at a constant rate of L_+/RC . (Note: capacitor is charged because the I_R current enters through the positive terminal of the capacitor for the polarity convention used)
- Due to virtual ground imposed by the left op amp on the positive terminal of the capacitor, the negative terminal of the capacitor starts decreasing linearly with a constant rate of $-L_+/RC$ as the capacitor voltage V_C increases.
- The voltage output V_{out1} is directly connected to the negative terminal of the capacitor and have the same behavior explained in the previous point and demonstrated by the following equations: (Note: supposing $V_{out1}(0) = V_{TH}$ at $t = 0$, corresponding to the time instant when V_{out2} starts to be at L_+)

$$I_C = C \frac{dV_C(t)}{dt} = -C \frac{dV_{out1}(t)}{dt} = \frac{L_+}{R} \rightarrow V_{out1}(t) = V_{out1}(0) - \int \frac{L_+}{R} dt = V_{TH} - \frac{L_+}{RC} t$$

- When the output voltage V_{out1} reaches the negative threshold voltage V_{TL} of the bistable circuit, the output voltage V_{out2} switches to L_- and the current through the resistor R becomes $I_R = L_-/R$ (changing sign and direction).
- The capacitor starts discharging at a constant rate of L_-/RC and the output voltage V_{out1} increases linearly with a slope of $-L_-/RC$ until it reaches the positive threshold voltage V_{TH} . At that point, the bistable circuit changes the output voltage V_{out2} back to L_+ and the cycle repeats indefinitely.

$$I_C = C \frac{dV_C(t)}{dt} = -C \frac{dV_{out1}(t)}{dt} = \frac{L_-}{R} \rightarrow V_{out1}(t) = V_{out1}(0) - \int \frac{L_-}{R} dt = V_{TL} - \frac{L_-}{RC} t$$

The period of the output signals is the time it takes for the output voltage V_{out1} to go from V_{TH} to V_{TL} (during T_1) and back to V_{TH} (during T_2).

$$\text{at } T_1, V_{out1} \text{ reaches } V_{TL} : V_{out1}(T_1) = V_{TH} - \frac{L_+}{RC} T_1 = V_{TL} \rightarrow T_1 = RC \frac{(V_{TH} - V_{TL})}{L_+}$$

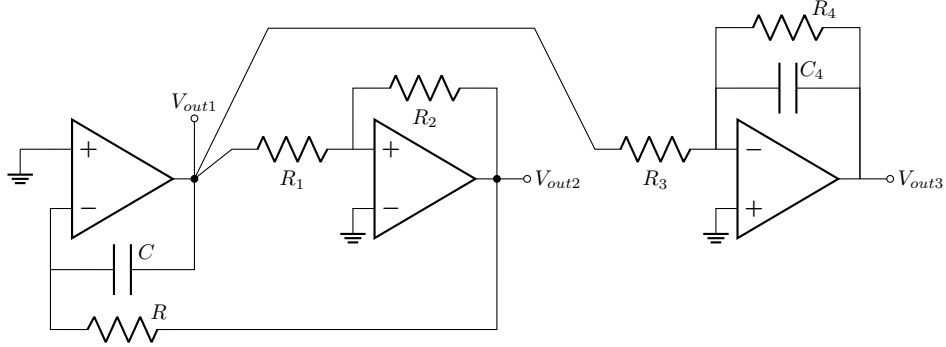
$$\text{at } T_2, V_{out1} \text{ reaches } V_{TH} : V_{out1}(T_2) = V_{TL} - \frac{L_-}{RC} T_2 = V_{TH} \rightarrow T_2 = RC \frac{(V_{TL} - V_{TH})}{L_-}$$

If the bistable circuit is symmetric, with $L_+ = L_-$, then the total period of the output signals is:

$$T = T_1 + T_2 = RC \frac{(V_{TH} - V_{TL})}{L_+} + RC \frac{(V_{TL} - V_{TH})}{L_-} = 2RC \frac{(V_{TH} - V_{TL})}{L_+}$$

6.3 Sine wave generator

By adding a bandpass filter to the triangular wave signal from the output of the astable multivibrator, it is possible to attenuate the harmonics of the signal and keep the fundamental sine wave. The bandpass filter can be a first-order low-pass filter implemented with an op amp. A first order low-pass filter may not be enough to obtain a good sine wave, because the third harmonic is attenuated to 30%, maybe a second order low-pass filter is needed.



The transfer characteristic of the low-pass filter is given by:

$$H(j\omega) = \frac{V_{out3}}{V_{in3}} = -\frac{Z_f}{Z_{in}} = -\frac{R_4 \cdot Z_{C4}}{R_4 + Z_{C4}} \cdot \frac{1}{R_3} = -\frac{R_4/j\omega C_4}{R_4 + 1/j\omega C_4} \cdot \frac{1}{R_3} = \underbrace{-\frac{R_4}{R_3}}_{\text{gain}} \cdot \underbrace{\frac{1}{1 + j\omega R_4 C_4}}_{\text{low-pass filter}}$$

$$|H(j\omega)_{\text{filter only}}| = \left| \frac{1}{\sqrt{1 + (\omega R_4 C_4)^2}} \right| = \frac{1}{\sqrt{2}} \quad \rightarrow \quad \omega = \frac{1}{R_4 C_4} \quad \rightarrow \quad f_c = \frac{1}{2\pi R_4 C_4}$$

7 Output stages

7.1 Introduction

Goal

Due to the maximum current limits of the op amps, they cannot be used to drive loads (speakers) directly. A power amplifier is needed to preserve the same signal waveform, but with higher current and power. A good power amplifier should have:

- support for large signal operation (10-20 V)
- low output impedance (to drive low impedance loads with high efficiency)
- low harmonic distortion (to preserve the signal waveform)

Power amplifiers classes

- class A: the output stage is always on, even without input signal
- class B: the output stage is on only for half of the input signal period
- class AB: the output stage is on for more than half of the input signal period
- class C: the output stage is on for less than half of the input signal period

THD - Total Harmonic Distortion

The total harmonic distortion (THD) is a measure of the quality of the output signal. It is defined as the ratio of the RMS power of the entire harmonic content (excluding the fundamental frequency) of the output signal to the RMS power of the fundamental frequency.

$$THD = \frac{\sqrt{V_2^2 + V_3^2 + V_4^2 + \dots}}{V_1} = \frac{\sqrt{\sum_{i \geq 2} V_i^2}}{V_1}$$

7.2 Class A - Emitter follower

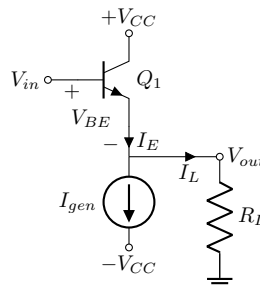
Circuit description

The simplest class A output stage is the emitter follower, which is a BJT transistor in which the input signal is applied to the base and the output is taken from the emitter. The collector is connected to the supply voltage. This configuration is called “emitter follower” because the output voltage in the emitter follows the input voltage at the base, with a fixed voltage drop V_{BE} . The transistor acts also as a current amplifier to drive the load with appropriate power.

$$V_{out} = V_{in} - V_{BE} \quad I_E = \beta I_B$$

To ensure that the transistor is always on, there must be a bias current flowing from the emitter, even without input signal. This can be achieved by using a current generator connected between the emitter and ground. The generator current I_{gen} must be higher than the maximum current $I_{L,max}$ that can be delivered to the load, to ensure that I_E current is always positive.

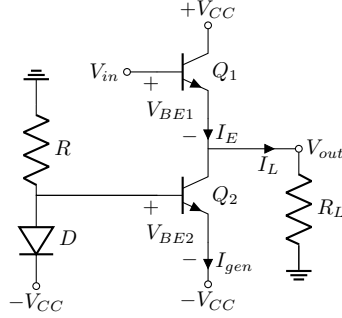
$$I_E = I_{gen} + I_L > 0 \quad \rightarrow \quad I_{gen} > |I_L|$$



With diode to bias transistor

It is possible implement the ideal current generator by using a diode with a fixed voltage drop V_D to impose a fixed voltage drop $V_{BE2} = V_D$ across the transistor Q_2 that generates the constant current I_{gen} .

The problem is that small variations on the voltage V_D (due to component heating) are exponentially amplified in current variations I_{gen} by the transistor Q_2 and the output stage becomes unstable.

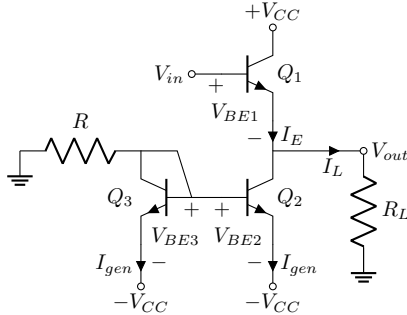


$$I_{gen} = I_{C2} = I_S \left(e^{V_{BE2}/V_T} - 1 \right)$$

With current mirror transistors

The idea is to use a current mirror to generate the bias current I_{gen} for the output stage. The current mirror is made of two matched transistors Q_2 and Q_3 that share the same base voltage $V_{BE2} = V_{BE3}$ and therefore the same working point. The current I_{gen} is set by the resistor R and the voltage drop V_{BE3} across the transistor Q_3 .

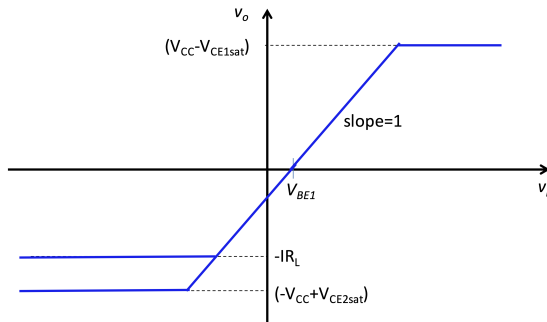
The problem of this configuration is that the current mirror transistor should be matched (should come from the same silicon wafer) and sometimes this requirement is impossible to achieve.



$$I_{gen} = I_R = \frac{V_R}{R} = \frac{|-V_{CC}| - V_{BE3}}{R}$$

Transfer characteristic

The transfer characteristic of the emitter follower is a straight line with a slope of 1 and a vertical offset of $-V_{BE1}$. The upper limit of the output voltage is $V_{CC} - V_{CE1sat}$ that occurs when the transistor Q_1 is in saturation. The lower limit of the output voltage is the minimum (in absolute value) between $-V_{CC} + V_{CE2sat}$ (saturation of Q_2) and $-I_{gen} \cdot R_L$ (maximum load current before Q_1 enters cutoff).



$$V_{out} = V_{in} - V_{BE1}$$

$$V_{out,max} = V_{CC} - V_{CE1sat}$$

$$V_{out,min} = \max \{ -V_{CC} + V_{CE2sat}, -I_{gen} \cdot R_L \}$$

Power dissipation

In normal operation, when the load is connected and $I_{gen} = V_{CC}/R_L$ (the maximum load current that absorb the load when the output voltage is at its maximum), the maximum power dissipated by the transistor Q_1 happens when the output voltage is zero. This is the normal resting point of the output stage and it is one of the most common working points. The transistor must be able to dissipate the power at this point for long periods of time without overheating.

$$V_{out} = 0 \text{ V} \quad V_{CE1} = V_{CC} \quad I_{CE} = I_{gen} \quad \rightarrow \quad P_{Q1,max} = V_{CC} \cdot I_{gen}$$

When the load is disconnected, the maximum power dissipated by Q_1 happens when:

$$V_{out} = -V_{CC} \quad V_{CE1} = 2V_{CC} \quad I_{CE} = I_{gen} \quad \rightarrow \quad P_{Q1,max,no-load} = 2V_{CC} \cdot I_{gen}$$

When the load is short-circuited, the current through Q_1 reach really high values (tends to infinity) and the Q_1 transistor can be destroyed by the high power dissipation. To prevent this, a short circuit protection can be implemented.

Power conversion efficiency

The power conversion efficiency η of an output stage is defined as the ratio between the power delivered to the load and the power drawn from the supply:

$$\eta = \frac{\text{load power } P_L}{\text{supply power } P_S}$$

For a class A output stage, the RMS power delivered to the load for a sinusoidal input signal is:

$$P_L = \frac{(V_{out}/\sqrt{2})^2}{R_L} = \frac{1}{2} \cdot \frac{V_{out}^2}{R_L}$$

The RMS power drawn from the supply is given by the sum of the power provided by the negative supply $I_{gen} \cdot V_{CC}$ and the power provided by the positive supply $(I_L + I_{gen}) \cdot V_{CC}$. For a sinusoidal input signal, the mean load current is $I_L = 0$, so the total power drawn from the supply is:

$$P_S = P_{S,negative} + P_{S,positive} = I_{gen} \cdot V_{CC} + (I_L + I_{gen}) \cdot V_{CC} = 2I_{gen} \cdot V_{CC}$$

The power conversion efficiency of a class A output stage is therefore:

$$\eta = \frac{P_L}{P_S} = \frac{1/2 \cdot V_{out}^2 / R_L}{2I_{gen} \cdot V_{CC}} = \frac{1}{4} \cdot \frac{V_{out}}{I_{gen} \cdot R_L} \frac{V_{out}}{V_{CC}} \leq 25\%$$

The maximum efficiency is achieved when $V_{out} = V_{CC}$ and $I_L = I_{gen} \rightarrow V_{out} = I_{gen} \cdot R_L$.

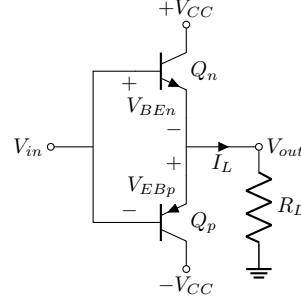
Advantages and disadvantages

- circuit is simple and easy to design, with really low distortion (THD $\approx 0.1\%$)
- low efficiency (max 25%) and high power dissipation makes it unsuitable for high power applications
- to avoid transistor saturation, output swing is limited and efficiency is even lower (10-20 %)

7.3 Class B - Push-pull stage

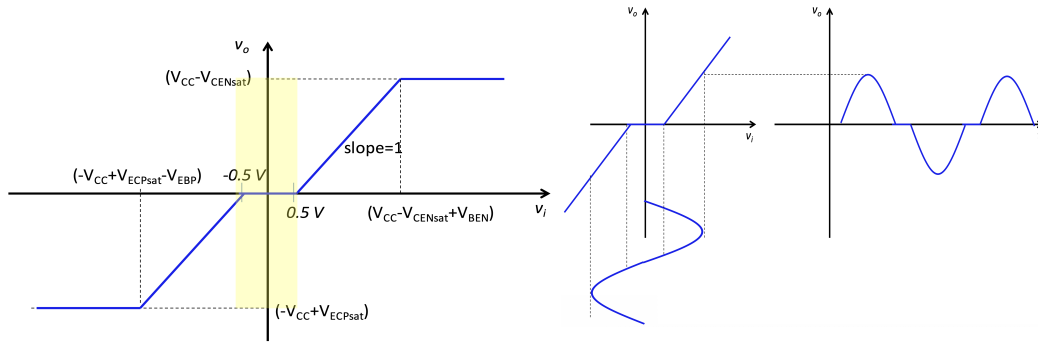
Circuit description

The idea of the class B output stage is to use two complementary transistors Q_n and Q_p in a push-pull configuration: one npn transistor drives the positive half of the input signal, while the other pnp transistor drives the negative half of the input signal and only one transistor is on at a time.



Transfer characteristic

The transfer characteristic of the class B output stage is a straight line with a slope of 1 with a dead zone called crossover or “notch” distortion between $-V_{EB,p}$ and $V_{BE,n}$.



Transfer characteristic of a class B output stage

Example of input and output signals (sinusoidal) of a class B output stage, the crossover distortion is clearly visible around the zero crossing of the output signal.

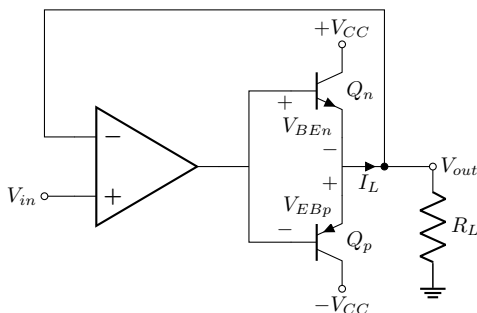
Power conversion efficiency

Can be demonstrated that the maximum power conversion efficiency of a class B output stage is:

$$\eta_{max} = \frac{\pi}{4} = 78.5\%$$

Crossover distortion

The crossover distortion is the distortion that occurs in the output signal when the input signal is around zero, due to the dead zone between $-V_{EB,p}$ and $V_{BE,n}$.



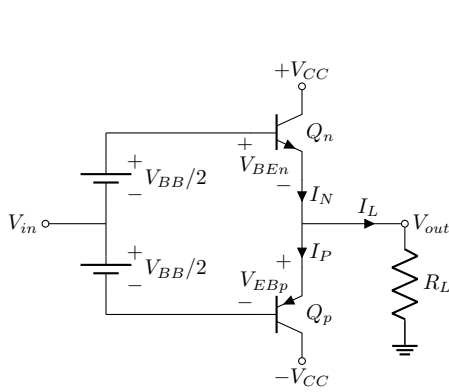
This distortion can be reduced by using an op amp to drive the input signal of the output stage with feedback loop to compensate the dead zone as shown on the left.

It is important to keep in mind op amps limitations, especially low slew rates can limit the compensation.

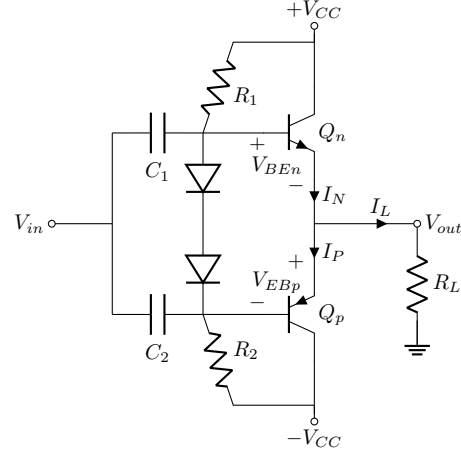
7.4 Class AB

Circuit description

The main idea of the class AB output stage is to reuse the same push-pull configuration of the class B output stage, but with a bias voltage V_{BB} applied to the bases of both transistors Q_n and Q_p to keep them slightly on even without input signal. In this way, transistors are always slightly conductive and are able to drive immediately the output signal without the class B delay and the crossover distortion is virtually eliminated.



Circuit of a class AB output stage with bias voltage V_{BB} applied via two ideal voltage sources.



Circuit of a class AB output stage with bias voltage V_{BB} applied via two diodes.

In real circuits, the bias voltage V_{BB} is imposed by using two diodes connected in series between the bases of the two transistors Q_n and Q_p as shown on the right. There are also two capacitors C_1 and C_2 to filter the bias voltage and two resistors R_1 and R_2 to properly limit the current through the diodes.

Circuit behavior

1. When the input signal passes from zero to positive, the Q_n transistor starts to conduct more current and V_{BE_n} increases. Due to the fact that $V_{BE_n} + V_{BE_p} = V_{BB}$, the V_{BE_p} decreases and the Q_p transistor starts to conduct less current until it reaches its minimum conduction.
2. Vice versa, when the input signal passes from zero to negative, the Q_p transistor starts to conduct more current and V_{BE_p} increases. The V_{BE_n} decreases and the Q_n transistor starts to conduct less current until it reaches its minimum conduction.

If the bias voltage V_{BB} is big enough, both transistors are slightly conductive at zero input signal and are able to immediately drive the output signal without delay eliminating the crossover distortion. If the bias voltage V_{BB} is too big, it is possible that both transistors remain conductive for the entire input signal period, and the output stage behaves like a class A output stage with high power dissipation and low efficiency.

Quiescent power dissipation

The problem of this configuration is that when both transistors are on at the same time, there is a direct current flowing from the positive supply to the negative supply through the transistors, that increases power dissipation and reduces efficiency.

If the V_{BB} is properly chosen, the quiescent current I_Q is smaller than the load current I_L , so the quiescent power dissipation is negligible and the efficiency is close to the maximum efficiency of a class B output stage.

Current analysis

From the diode equations applied to the npn and pnp transistors:

$$I_N = I_S \left(e^{V_{BE_n}/V_T} - 1 \right) \approx I_S \left(e^{V_{BE_n}/V_T} \right) \rightarrow V_{BE_n} = V_T \ln \left(\frac{I_N}{I_S} \right)$$

$$I_P = I_S \left(e^{V_{EB_p}/V_T} - 1 \right) \approx I_S \left(e^{V_{EB_p}/V_T} \right) \rightarrow V_{EB_p} = V_T \ln \left(\frac{I_P}{I_S} \right)$$

$$I_Q = I_S \left(e^{V_{BB}/2V_T} - 1 \right) \approx I_S \left(e^{V_{BB}/2V_T} \right) \rightarrow V_{BB} = 2V_T \ln \left(\frac{I_Q}{I_S} \right)$$

$$V_{BE_n} + V_{EB_p} = V_{BB} \rightarrow V_T \ln \left(\frac{I_N}{I_S} \right) + V_T \ln \left(\frac{I_P}{I_S} \right) = 2V_T \ln \left(\frac{I_Q}{I_S} \right) \rightarrow I_N \cdot I_P = I_Q^2$$

From the Kirchhoff's current law at the output node:

$$I_N = I_P + I_L$$

Putting both equations together:

$$I_N^2 - I_L \cdot I_N - I_Q^2 = 0$$

By solving the quadratic equation, we can find the current I_N and I_P :

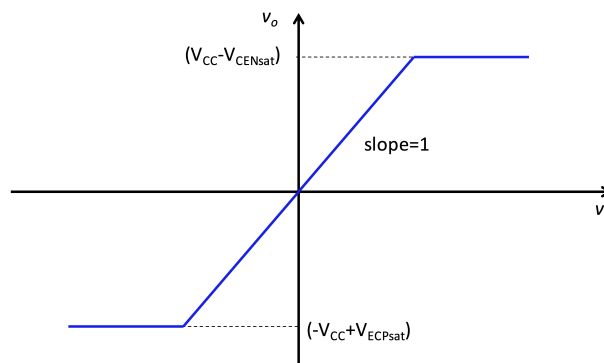
$$I_N = \frac{I_L}{2} + \sqrt{\left(\frac{I_L}{2} \right)^2 + I_Q^2} \quad I_P = -\frac{I_L}{2} + \sqrt{\left(\frac{I_L}{2} \right)^2 + I_Q^2}$$

These results show that:

- if $I_L = 0$, then $I_N = I_P = I_Q$ and both transistors are slightly conductive
- if I_L is big enough, then $I_N \approx I_L$ and $I_P \approx 0$ (or vice versa if I_L is negative), by neglecting the I_Q^2 term in the square root, showing that the output stage behaves like a class B output stage when the load current is big enough

Transfer characteristic

The transfer characteristic of the class AB output stage with ideal bias voltage V_{BB} is a perfect straight line with a slope of 1, no crossover distortion and no voltage offset.



Disadvantages

The main problem of the class AB output stage is to properly set the bias voltage V_{BB} to eliminate crossover distortion without increasing too much the quiescent power dissipation. This is a really hard task in real circuits because of the components tolerances and the temperature variations.

8 Arduino

8.1 Introductions to Arduino and microcontrollers

Microcontrollers

Microcontrollers are complete computer systems with microprocessor (CPU), clock oscillator, memory and I/O peripherals integrated on a single chip. Microcontrollers are used to get data from the environment through sensors, process it and control actuators accordingly. Due to memory and processing power limitations, code for microcontrollers must be written in a very efficient way.

Arduino Uno

Arduino Uno is a microcontroller board based on the ATmega328P microcontroller. It has 14 digital input/output pins and 6 analog input pins and a 16 MHz quartz crystal. It has an AREF pin for analog reference voltage for ADC and a reset button. It has a USB connection for programming and a power jack for external power supply. It has a 5V pin (400 mA) and a 3.3V pin (50 mA) connected to the onboard voltage regulator and a Vin pin connected directly to the input power supply source.

The Atmega328P microcontroller has 32 KB of ISP flash memory for storing code, 2 KB of SRAM for local variables and 1 KB of EEPROM for non volatile data. It has 23 GPIO (general purpose input/output), 32 general purpose working registers, 6 channels 10-bit ADC, Serial UART, SPI and I²C communication interfaces. It has three timers, internal and external interrupts and a watchdog timer. It operates at a voltage between 1.8 and 5.5 V.

Arduino Mega

Arduino Mega is a microcontroller board based on the ATmega2560 microcontroller. It has 54 digital input/output pins and 16 analog input pins and a 16 MHz quartz crystal. Due to the higher number of pins, it is used for more complex projects that require more I/O peripherals.

Arduino Due

Arduino Due is a microcontroller board based on the Atmel SAM3X8E ARM Cortex-M3 CPU. It has 54 digital input/output pins and 12 analog input pins and a 84 MHz quartz crystal. Due to the higher processing power, it is used for more complex projects that require more processing power and more I/O peripherals. **It operates at a voltage of 3.3 V and it is not 5V tolerant.**

Arduino Nano 33 BLE Sense Rev2

Arduino Nano 33 BLE Sense Rev2 is a microcontroller board based on the Nordic Semiconductor nRF52840 ARM Cortex-M4 CPU in the same form factor as the Arduino Nano. It has 14 digital input/output pins, 8 analog input pins and a 64 MHz quartz crystal. It has a Bluetooth Low Energy (BLE) module and a wide range of sensors, including an accelerometer, a gyroscope, a magnetometer, a temperature sensor, a humidity sensor, a pressure sensor and a microphone and has an RGB LED. **It operates at a voltage of 3.3 V and it is not 5V tolerant.**

Other boards

- Arduino Uno Wifi Rev2: same form factor of the standard Arduino Uno but with an ATmega4809 microcontroller and a WiFi module with integrated TCP/IP protocol stack.
- Arduino STAR - Otto: equipped with a powerful ARM processor to control power demanding peripherals like LCD, Audio, Camera, etc.
- Arduino AlicePad: board for wearables and clothing
- Arduino Nano: same microcontroller as the Uno but in a smaller form factor
- Arduino Giga R1 Wifi and Arduino Portenta H7: boards with an ARM Cortex-M7 microcontroller and a WiFi module, for high performance applications like robotics, AI and machine learning
- Shields: passive boards that can be plugged on top of an Arduino board to extend its functionality

8.2 Programming on Arduino - pin modes, interrupts and timers

Bootloader

The bootloader is a small program stored in a private section of the flash memory that allows the microcontroller to be programmed through a serial communication protocol. It is responsible also for the execution of the user program and for the management of the reset and power-on events.

To make programming process easier, Arduino boards come also with a USB-to-serial converter chip that interfaces the bootloader with the USB communication protocol used by computers. This allows the microcontroller to be programmed with a computer.

Without a bootloader, the only way to program the microcontroller is through the SPI communication pins, which requires an external programmer board and a more complex programming process.

Program structure

An arduino program is called a sketch and it is written in C/C++. It has two main functions:

- **setup()**: executed only once, when the microcontroller is powered on or reset; it is used to initialize variables, object instances of peripherals, pin modes and other settings
- **loop()**: executed repeatedly after the **setup()** function; it is used to implement the main logic of the program

A good practice is to prefer global variables rather than local variables, because first ones are stored in the flash memory and do not consume SRAM, which is a very limited resource on microcontrollers.

Pin modes and pin functions

Arduino pins can be configured in different modes and can perform different functions.

- **digital output**: pin configured as a digital output, used to write digital signals (LOW or HIGH) through **digitalWrite(PIN, [HIGH,LOW])** function
- **digital input**: pin configured as a digital input, used to read digital signals (HIGH or LOW) through **digitalRead(PIN)** function
- **analog output**: pin configured as a digital PWM output, used to write PWM signals with custom duty cycle in range 0-255 through **analogWrite(PIN, value)** function, PWM frequency can vary depending on board and pin used
- **analog input**: pin configured as an analog input, used to read analog signals through the ADC and get a digital value in range 0-1023 (for 10-bit ADC) through **analogRead(PIN)** function

To configure how the pin behaves, the **pinMode(PIN, MODE)** function is used in the **setup()** function, where **MODE** can be one of the following:

- **OUTPUT**: configures the pin as a digital output
- **INPUT**: configures the pin as an analog/digital input
- **INPUT_PULLUP**: configures the pin as a digital input with an internal pull-up resistor, in this mode the pin is connected to V_{CC} through a $\approx 20\text{ k}\Omega$ pull-up resistor, so the pin reads **HIGH** when it is not connected to ground and **LOW** when it is connected to ground.

Analog readings and ADC configuration

The resolution of the ADC can be changed by calling the **analogReadResolution(bits)** function in the **setup()** function. By default, the resolution is 10 bits (0-1023), but it can be increased to 12 bits (0-4095) on some boards like the Arduino Due and the Arduino Nano 33 BLE Sense Rev2.

It is possible to set an external high-voltage reference for the ADC by connecting that reference voltage to the AREF pin and calling the **analogReference(EXTERNAL)** function in the **setup()** function. By default, the reference voltage is the supply voltage (5V or 3.3V depending on the board).

The resolution step of the ADC can be calculated as follows:

$$\text{ADC step} = \frac{V_{REF}}{2^{\text{bits}}}$$

for Arduino Uno:	$\text{ADC step} = \frac{5\text{ V}}{2^{10}\text{ bits}} = 4.88\text{ mV}$
for Arduino Nano 33 BLE Sense:	$\text{ADC step} = \frac{3.3\text{ V}}{2^{12}\text{ bits}} = 0.806\text{ mV}$

The millis, micros and delay functions

These three functions are used to manage time in Arduino sketches and use internal timers of the microcontroller.

- `millis()`: returns the number of milliseconds since the microcontroller was powered on or reset; it returns an unsigned 32-bit integer, so it overflows after approximately 50 days
- `micros()`: returns the number of microseconds since the microcontroller was powered on or reset; it returns an unsigned 32-bit integer, so it overflows after approximately 70 minutes; due to limited clock speed on some boards, the resolution of this function can be 4 microseconds (16 MHz clock)
- `delay(ms)`: blocks the execution of the program for a specified number of milliseconds

Use of the `millis()` and `micros()` functions is preferred to the use of the `delay()` function, because it does not block the execution of the program and allows to implement non-blocking delays and timeouts, which are essential for implementing real-time applications.

Interrupts

An interrupt is a signal that can be generated by an external event (for example a button press) or by an internal event (for example a timer overflow) that interrupts the normal execution of the program and executes a special function called interrupt service routine (ISR). Interrupts are used to handle events that require immediate attention and cannot be handled in the loop function of the program, for example user interactions.

Interrupt service routines has the following characteristics:

- they must be as short and fast as possible
- only one interrupt can be executed at a time so if multiple interrupts are generated at the same time, they are executed in order of priority
- only `delayMicroseconds()` function works inside an ISR, while `delay()` and `millis()` functions do not work because they rely on interrupts to update the internal timers
- they do not accept parameters and do not return values, so they must use `volatile` global variables to communicate with the main program. The keyword `volatile` is used to prevent the compiler from optimizing operations on those variables and causing race conditions.

To attach an interrupt to a pin, the `attachInterrupt(digitalPinToInterrupt(PIN), ISR, MODE)` function is used in the `setup()` function where:

- `PIN` is the pin number to which the interrupt is attached (only supported pins can be used)
- `ISR` is the name of the interrupt service routine function
- `MODE` is the condition that triggers the interrupt and can be one of the following:
 - `RISING`: triggers the interrupt when the pin goes from `LOW` to `HIGH`
 - `FALLING`: triggers the interrupt when the pin goes from `HIGH` to `LOW`
 - `CHANGE`: triggers the interrupt when the pin changes state (from `LOW` to `HIGH` or vice versa)
 - `LOW`: triggers the interrupt when the pin is `LOW` (only for some boards)
 - `HIGH`: triggers the interrupt when the pin is `HIGH` (only for some boards)

Switch-case, break continue statements

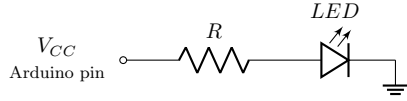
Switch-case statements are used to execute different blocks of code based on the value of a variable. They are more efficient than if-else statements when there are many cases to handle, because they use a jump table instead of evaluating multiple conditions.

The `break` and `continue` statements are used to control the flow of loops and switch-case statements. The `break` statement is used to exit a loop or a switch-case statement, while the `continue` statement is used to skip the current iteration of a loop and continue with the next iteration.

8.3 Simple circuits with Arduino

Drive a LED directly connected to an Arduino pin

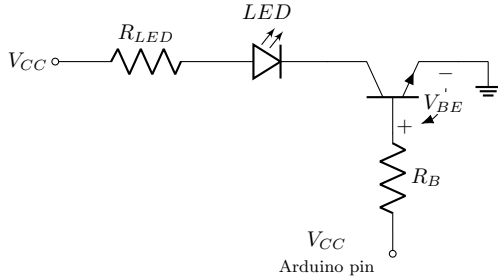
To correctly bias a LED, a current limiting resistor must be used in series with the LED. The value of the resistor can be calculated using Ohm's law as follows.



$$R = \frac{V_R}{I_R} = \frac{V_{CC} - V_{LED}}{I_{LED}}$$

Drive high current into a LED with a BJT

Arduino pins can source or sink a limited amount of current (4mA for Nano 33 BLE Sense Rev2). If a higher current is needed, a BJT can be used as a switch to drive the LED. The base of the BJT is connected to the Arduino pin through a current limiting resistor, the collector is connected to the LED with a current limiting resistor in series and the emitter is connected to ground. The transistor is used as a switch in saturation regime, so the voltage drop across the collector-emitter junction is negligible and the current through the LED is determined by the current limiting resistor and the supply voltage.



$$R_{LED} = \frac{V_{R_{LED}}}{I_{R_{LED}}} = \frac{V_{CC} - V_{LED}}{I_{LED}}$$

$$R_B = \frac{V_{R_B}}{I_{R_B}} = \frac{V_{CC} - V_{BE}}{I_B}$$

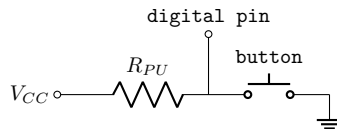
Dimming an LED / RGB-LED with analogWrite() and PWM signal

By using the `analogWrite(PIN, value)` function, it is possible to generate a PWM signal on a digital pin (with PWM support) and control the brightness of a LED by changing the duty cycle of the PWM signal. Value can be in range 0-255, where 0 means 0% duty cycle (LED off) and 255 means 100% duty cycle (LED fully on).

On the Arduino Nano 33 BLE Sense Rev2, there is a built-in RGB LED controlled by three virtual pins (LED_R, LED_G, LED_B). By using the `analogWrite()` function on these pins, it is possible to control the brightness of each color and create different colors by mixing the three primary colors.

Read a button state with digitalRead(), pull-up resistors and debouncing

A button is a simple switch that can be used to connect a pin to ground when pressed. The main problem is that when the button is not pressed, the pin is left floating and can read random values. To solve this problem, a pull-up resistor should be used to connect the pin to V_{CC} . In this way, when the button is not pressed, the pin reads HIGH and when the button is pressed, the pin reads LOW.



To choose the value of the pull-up resistor, it is necessary to consider the current that flows through the resistor when the button is pressed (when the button is not pressed, the current is negligible because the pin is in high impedance state). The suggested current is in the order of 0.1-0.2 mA, so the value of the pull-up resistor can be calculated as follows.

$$I_{R_{PU}} = \frac{V_{CC}}{R_{PU}} \quad \text{common values are :} \quad \begin{array}{l} 20 - 22 \text{ k}\Omega \text{ for } 5 \text{ V boards} \\ 10 - 12 \text{ k}\Omega \text{ for } 3.3 \text{ V boards} \end{array}$$

On some boards like the Arduino Uno, there are internal pull-up resistors of 20 kΩ that can be enabled by configuring the pin mode as `INPUT_PULLUP`. In this way the button can be connected directly between the pin and ground, without the need of an external pull-up resistor.

Usually, when a button is pressed, the transition from **HIGH** to **LOW** is not clean and immediate, but there are some oscillations called bouncing due to mechanical switch contacts oscillations. To solve this problem, a simple software debouncing technique can be used, which consists in leaving a short time (10 and 50 ms) between two consecutive readings of the button state.

Alternatively, a pull-down resistor can be used between the pin and ground, and the button will be connected between the pin and V_{CC} . The button behavior becomes active **HIGH**, meaning the state will be **HIGH** when the button is pressed and **LOW** when the button is not pressed.

Read a potentiometer value with `analogRead()`

It is possible to read the voltage value of a potentiometer wiper terminal by connecting it to an analog input pin. The voltage at the wiper terminal will be between 0 V and V_{CC} and can be read by an analog pin using the `analogRead()` function. The value returned is in range 0-1023 for a 10-bit ADC or in range 0-4095 for a 12-bit ADC and is linearly proportional to the wiper voltage.

In some applications, it is necessary to scale the value read from the potentiometer to a different range (for example in range 0-255 to control the brightness of an LED). To do this, it is possible to use the `map(x, in_min, in_max, out_min, out_max)` function to map the value `x` from the input range `[in_min, in_max]` to the output range `[out_min, out_max]` as follows:

$$\text{map}(x, \text{in_min}, \text{in_max}, \text{out_min}, \text{out_max}) = \frac{(x - \text{in_min}) \times (\text{out_max} - \text{out_min})}{\text{in_max} - \text{in_min}} + \text{out_min}$$

Read analog sensors with `analogRead()`

Sometimes the output voltage of an analog sensor has a smaller range $[0, V_{\text{sensor,max}} < V_{CC}]$. To correctly read the sensor value, it is necessary to scale the sensor output voltage to the full range of the ADC. This can be done in different ways:

- using the `map()` function to map the value read from the sensor, but does not increase the resolution of the ADC because is done after the ADC conversion
- using an external op-amp to amplify the sensor output voltage to the full range of the ADC; attention must be paid to not exceed the maximum input voltage of the ADC
- using the `AREF` pin to set the reference voltage of the ADC to the maximum sensor output voltage, so the ADC will automatically scale the sensor output voltage to the full range of the ADC (this one is the preferred solution)

Read temperature with analog TMP36 temperature sensor

TMP36 is an analog temperature sensor that outputs a voltage linearly proportional to the temperature. The output voltage has a 0.5 V offset (0.5 V correspond to 0°C) and a slope of 10 mV/°C. To read the temperature with an Arduino, the output voltage of the TMP36 sensor can be read with an analog input pin and converted into temperature using the following mapping formula considering (155 correspond to 0.5 V and 330 is the temperature range for 3.3 V power supply).

$$T = \frac{V_{\text{out}} - 0.5 \text{ V}}{10 \text{ mV}/^{\circ}\text{C}} = (V_{\text{out}} - 0.5 \text{ V}) \times 100 \text{ }^{\circ}\text{C}/\text{V} \quad \rightarrow \quad \text{temperature} = (\text{value} - 155) * 330 / 1023;$$

Read humidity with analog HIH5030 humidity sensor

HIH5030 is an analog humidity sensor that outputs a voltage linearly proportional to the relative humidity. The output voltage can be read with an analog input pin and converted into relative humidity using one of the following mapping formulas (from the datasheet).

$$\begin{aligned} RH &= (V_{\text{out}}/V_{CC} - 0.1515)/0.00636 \\ V_{\text{out}} &= \text{measured_value} \cdot V_{CC}/1023 \end{aligned} \quad \rightarrow \quad \text{humidity} = (\text{value} / 1023 - 0.1515) / 0.00636;$$

8.4 Asynchronous Serial Protocol

Introduction to asynchronous serial protocols

Asynchronous Serial Protocol is a family of communication protocols that allow to exchange data between the microcontroller and other devices using a serial communication interface. It has the following characteristics:

- asynchronous: the data is transmitted without a clock signal, so the sender and the receiver must agree on the baud rate (bits per second) and the format of the data (start and stop bits, parity bit, etc.) before the start of the communications
- serial: the data is transmitted serially, one bit at a time, not in parallel

UART - Universal Asynchronous Receiver-Transmitter protocol

The UART protocol is the asynchronous serial protocol used for the communication between an Arduino board and other device. It is a full-duplex protocol that uses two lines for communication: TX (transmit) and RX (receive). The TX line of one device is connected to the RX line of the other device and vice versa. In idle state, lines are **HIGH**. The communication has the following format:

- start bit: 1 bit **LOW**, indicates the start of a data frame
- data bits: bits transmitted in little endian order (least significant bit first)
- stop bit: 1 bit **HIGH**, indicate the end of a data frame

Serial communication with Arduino

In Arduino boards, the UART protocol is implemented on the digital pins 0 (RX) and 1 (TX) for communication with other devices and on the USB port for communication with the computer via the USB-to-serial converter. The Arduino IDE provides a built-in Serial Monitor or a Serial plotter to visualize the data sent from the Arduino to the computer through the USB port and to send data from the computer to the Arduino.

In Arduino sketches, the UART protocol is accessed through the `Serial` class, which provides the following functions:

- `Serial.begin(baud_rate)`: initializes the serial communication with the specified baud rate, usually used in the `setup()` function, common baud rates are 300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, 115200 bps
- `Serial.print(data)`: sends data to the serial port as human-readable ASCII text
- `Serial.println(data)`: same as `Serial.print()`, plus append a newline character at the end
- `Serial.available()`: returns if there is data available to read from the serial port
- `Serial.read()`: reads a byte of data from the serial port; returns -1 if no data is available

8.5 Serial Peripheral Interface - SPI

Introduction to SPI protocol

Serial Peripheral Interface (SPI) is a synchronous serial communication protocol that uses a master-slave configuration to exchange data between a single controller device (master) and one or more controlled peripheral devices (slaves).

SPI connection lines

The SPI protocol uses at least four lines for communication:

- SCLK (Serial Clock): generated by the master to synchronize the communication
- CPO (Controller In Peripheral Out): transmit data from the peripheral to the controller (ex MISO)
- COPI (Controller Out Peripheral In): transmit data from the controller to the peripheral (ex MOSI)
- CS (Chip Select): line used to select a specific peripheral device (active **LOW**)

SCLK, CPO and COPI lines are shared between all devices on the SPI bus, while each peripheral device has its own CS line connected to the master.

SPI communication process

The SPI communication works as follows:

- the master selects the slave device by pulling its CS line **LOW**
- the master generates the clock signal on the SCLK line
- on each falling edge of the clock signal, the master and the slave transmit one bit of data on the CPO and COPI lines that will be read during the next clock period
- after the communication is finished, the master stops the clock signal and pulls the CS line **HIGH**

8.6 Libraries in Arduino

Introduction to Arduino/C/C++ libraries

A library is a collection of functions and declarations that provide extra functionality which can be used in multiple arduino sketches without the need to write the code from scratch. Libraries are organized in:

- header file (.h): contains the declarations of the functions and variables provided by the library
- source file (.cpp): contains the implementation of the functions declared in the header file

Classes and objects in Arduino libraries

Usually libraries contain classes that are user-defined data structures that encapsulate data and functions and are used to manage peripherals and sensors or software tools in a more organized way. Classes are defined in the header file and implemented in the source file. Classes are instantiated in the arduino sketch to create objects of that class, which can be used to call the functions provided by the library and manage the peripheral or sensor associated with that class. Classes have the following components:

- class constructor: a special function that is called when an object of the class is instantiated, it initializes the object and its variables
- public functions: functions that can be called from outside the class, they are used to provide the functionality of the class
- private variables: variables that can only be accessed from within the class, they are used to store the state of the object and other internal data
- private functions: functions that can only be called from within the class, they are used to implement internal functionality of the class that is not exposed to the user, conventionally they are prefixed with an underscore to indicate that they are private

Example of header file with class definition

```
1  #ifndef CLASS_NAME_H // include guard to prevent multiple inclusions
2  #define CLASS_NAME_H // of the header file
3
4  #include <Arduino.h> // Arduino library with Arduino-specific functions and types
5
6  class ClassName {
7      public:
8          ClassName(); // constructor
9          void function1(); // public function or procedure
10         int function2(int param); // public function with parameter and return value
11
12         private:
13             int variable1; // private variable
14             float variable2; // private variable
15             void privateFunction(); // private function
16 };
17
18 #endif // CLASS_NAME_H
```

Example of source file with class implementation

```
1  #include <Arduino.h> // Arduino library with Arduino-specific functions and types
2  #include "ClassName.h" // header file with class definition
3
4  ClassName::ClassName() {
5      // constructor implementation
6  }
7
8  void ClassName::function1() {
9      // function implementation
10 }
11
12 int ClassName::function2(int param) {
13     // function implementation
14 }
15
16 void ClassName::privateFunction() {
17     // private function implementation
18 }
```

Library file organization and syntax highlighting

To use a user-made library in an Arduino sketch, the library files (header and source files) must be placed in a specific folder with the same name as the library in the Arduino libraries folder.

Usually, user-defined libraries contains symbols and variables that are not recognized by the Arduino IDE as keywords. To enable syntax highlighting for those symbols, it is possible to add a keyword file in the library folder with the same name as the library and the extension .txt that contains the list of keywords to be highlighted, for example:

```
1  --- inside ClassName.txt file ---
2  ClassName      KEYWORD1
3  function1      KEYWORD2
4  function2      KEYWORD2
```

8.7 I²C Protocol

Introduction to I²C Protocol

The I²C (Inter-Integrated Circuit) is a synchronous serial communication protocol that uses a master-slave configuration where multiple devices can communicate over a common bus made of only two lines.

I²C connection lines

The I²C protocol uses two lines for communication:

- SDA (Serial Data): line used to transmit data between devices
- SCL (Serial Clock): line used to synchronize the communication, is generated by the master device and can be forced low by the slave devices to pause the communication when they are not ready to receive or transmit data

NOTE: SDA line can change state only when SCL is LOW, except for the start and stop conditions.

I²C communication process

Every device on the I²C bus has a unique 7-bit address that is used to identify the device during communication. If the datasheet of the device specifies an 8-bit address, a right shift of 1 bit must be applied (the least significant bit is eliminated). The communication process works as follows:

- the master device initiates the communication by sending a start condition (SCL keeps HIGH while SDA goes from HIGH to LOW) and starts generating the clock signal on the SCL line
- the master device sends the 7-bit address of the slave device it wants to communicate with, followed by a R/W bit (0 for write, 1 for read) to indicate if it wants to send or receive data from the slave and in the end the master releases the SDA line to allow the slave device to respond
- the slave device with the matching address responds with an ACK bit (0) if it is ready to communicate, otherwise it responds with a NACK bit (1) if it is not ready
- if the slave device ACKs the address, the master and the slave can exchange data bytes, according to the R/W bit sent by the master; after each byte of data, the receiver must send an ACK bit (0) or a NACK bit (1) to indicate whether transmission was successful or not
- after the communication is finished, the master sends a stop condition (SCL keeps HIGH while SDA goes from LOW to HIGH) and stops generating the clock signal on the SCL line

Wire.h library to handle I²C communications

In Arduino, the I²C protocol is implemented through the `Wire` library, which provides functions to easily manage I²C communication. The library provides the following functions:

- `Wire.begin()`: initializes the I²C communication, must be called in the `setup()` function before using any other function of the library;
- `Wire.beginTransmission(address)`: initiates a transmission to the slave device with the specified address
- `Wire.write(data)`: enqueues a byte of data to the buffer for the slave device
- `Wire.endTransmission()` sends the bytes enqueued in the buffer and ends the transmission
- `Wire.requestFrom(address, quantity)`: requests a specified quantity of bytes from the slave device with the specified address; returns the number of bytes received from the slave device
- `Wire.read()`: reads a byte of data received from the slave device; returns -1 if no data is available

The I²C buffer is limited to 32 bytes, so if more data needs to be sent or received, it is necessary to split the data into multiple transmissions.

There can be multiple `Wire` buses on the same Arduino board, for example on the Arduino Nano 33 BLE Sense Rev2 there is a second I²C bus connected by `Wire1.h` library, specifically designed to manage the built-in sensors on the board.

Using DS1621 digital temperature sensor with I²C communication

The DS1621 is a digital temperature sensor that communicates over the I²C bus. It has a 7-bit address that can be partially configured using external pins of the sensor obtaining the 7-bit address in the following format: “1 0 0 1 A2 A1 A0”. I²C commands for the DS1621 sensor are as follows:

- 0xAC: write the next byte into the configuration register of the sensor
- 0x02: enable the continuous conversion mode of the sensor (temperature is continuously measured and updated in the sensor registers)
- 0xEE: start a single temperature conversion (temperature is measured once and updated in the sensor registers)
- 0xAA: read the temperature value from the sensor registers

Temperature values are represented in two bytes: the first byte contains the integer part of the temperature in two's complement format, while the second byte contains the fractional part of the temperature in increments of 0.5°C (0 for +0°C and 128 for +0.5°C).

The following sketch shows how to use the Wire library to read the temperature value from the DS1621 sensor and print it to the serial monitor.

```
1  #include <Wire.h> // I2C communication library
2
3  #define DS1621_ADDR 0x48 // I2C address of the DS1621 sensor when A2, A1, A0 are LOW
4  #define DS1621_CONFIG_REG 0xAC // configuration register address
5  #define DS1621_CONTINUOUS_CONV_CMD 0x02 // enable continuous conversion mode
6  #define DS1621_START_CONV_CMD 0xEE // start temperature conversion
7  #define DS1621_READ_TEMP_CMD 0xAA // read temperature value
8
9  void setup() {
10     Wire.begin(); // initialize I2C communication
11
12     Wire.beginTransmission(DS1621_ADDR); // start transmission to the sensor
13     Wire.write(DS1621_CONFIG_REG); // write to configuration register
14     Wire.write(DS1621_CONTINUOUS_CONV_CMD); // enable continuous conversion mode
15     Wire.endTransmission(); // end transmission
16
17     Wire.beginTransmission(DS1621_ADDR); // start transmission to the sensor
18     Wire.write(DS1621_START_CONV_CMD); // start temperature conversions
19     Wire.endTransmission(); // end transmission
20
21     Serial.begin(9600); // initialize serial communication for debugging
22 }
23
24 void loop() {
25     Wire.beginTransmission(DS1621_ADDR); // start transmission to the sensor
26     Wire.write(DS1621_READ_TEMP_CMD); // send command to read temperature
27     Wire.endTransmission(); // end transmission
28
29     Wire.requestFrom(DS1621_ADDR, 2); // request 2 bytes of data from the sensor
30
31     byte byte1 = Wire.read(); // read the integer part of the temperature
32     byte byte2 = Wire.read(); // read the fractional part of the temperature
33
34     // compute the temperature value
35     float temp = byte1 + (byte2 == 128 ? 0.5 : 0);
36
37     // print temperature value to serial monitor
38     Serial.println("Temperature: " + temp + " C");
39
40     delay(1000); // wait before reading the temperature again
41 }
```

Using TSL2561 light sensor with I²C communication

The TSL2561 is a digital light sensor that communicates over the I²C bus. It is equipped with two photodiodes on the same CMOS chip that measure the visible and infrared light intensity with 16-bit resolution. There are two independent ADC channels (0 and 1) to store respectively the visible light intensity and the infrared light intensity.

Values in the two channels registers are updated every 402ms. It is possible to further customize the integration time and the gain of the sensor with additional commands (see datasheet).

The sensor has a 7-bit address that can be partially configured using an external pin of the sensor (0x29 when LOW, 0x39 when floating, 0x49 when HIGH). I²C commands for the TSL2561 sensor are:

- 0x00: write the next byte into the command register of the sensor
- 0x03: power on the sensor (written on the command register)
- 0x00: power off the sensor (written on the command register)
- 0x81: access timing register of the sensor
- 0x02: set low gain (1x) with 402ms integration time (written on the timing register)
- 0x8C, 0x8D: read channel 0 LSB (least significant byte) and MSB (most significant byte)
- 0x8E, 0x8F: read channel 1 LSB (least significant byte) and MSB (most significant byte)

The following sketch shows how to use the Wire library to read the light intensity values from the TSL2561 sensor and print it to the serial monitor.

```
1  #include <Wire.h> // I2C communication library
2
3  #define TSL2561_ADDR 0x29 // I2C address of the TSL2561 sensor when ADDR_PIN is LOW
4  #define TSL2561_CMD_REG 0x00 // command register address
5  #define TSL2561_POWER_ON 0x03 // power on the sensor
6  #define TSL2561_POWER_OFF 0x00 // power off the sensor
7  #define TSL2561_TIMING_REG 0x81 // timing register address
8  #define TSL2561_LOW_GAIN_402MS 0x02 // set low gain with 402ms integration time
9  #define TSL2561_READ_CH0_LSB 0x8C // read channel 0 LSB
10 #define TSL2561_READ_CH0_MSB 0x8D // read channel 0 MSB
11 #define TSL2561_READ_CH1_LSB 0x8E // read channel 1 LSB
12 #define TSL2561_READ_CH1_MSB 0x8F // read channel 1 MSB
13
14 void setup() {
15     Wire.begin(); // initialize I2C communication
16
17     Wire.beginTransmission(TSL2561_ADDR); // start transmission to the sensor
18     Wire.write(TSL2561_CMD_REG); // write to command register
19     Wire.write(TSL2561_POWER_ON); // power on the sensor
20     Wire.endTransmission(); // end transmission
21
22     Wire.beginTransmission(TSL2561_ADDR); // start transmission to the sensor
23     Wire.write(TSL2561_TIMING_REG); // write to timing register
24     Wire.write(TSL2561_LOW_GAIN_402MS); // set low gain with 402ms integration time
25     Wire.endTransmission(); // end transmission
26
27     Serial.begin(9600); // initialize serial communication for debugging
28 }
29
30 void loop() {
31     Wire.beginTransmission(TSL2561_ADDR); // start transmission to the sensor
32     Wire.write(TSL2561_READ_CH0_LSB); // send command to read channel 0 LSB
33     Wire.endTransmission(); // end transmission
34     Wire.requestFrom(TSL2561_ADDR, 1); // request 1 bytes of data from the sensor
35     byte ch0_lsb = Wire.read(); // read channel 0 LSB
36
37     Wire.beginTransmission(TSL2561_ADDR); // start transmission to the sensor
38     Wire.write(TSL2561_READ_CH0_MSB); // send command to read channel 0 MSB
39     Wire.endTransmission(); // end transmission
40     Wire.requestFrom(TSL2561_ADDR, 1); // request 1 bytes of data from the sensor
41     byte ch0_msb = Wire.read(); // read channel 0 MSB
42
43     float lum = 256* ch0_msb + ch0_lsb; // compute lux value from channel values
44     Serial.println("Lux: " + lum + " lx"); // print lux value to serial monitor
45     delay(1000); // wait before reading the light intensity again
46 }
```

8.8 Adafruit HX8357 LCD TFT display

Adafruit HX8357 is an LCD TFT display with a resolution of 480x320 pixels with touchscreen capabilities. It can be controlled via SPI communication and there is an Arduino library provided by Adafruit that allows to easily control the display.

Connections

To connect the display to an Arduino board via SPI, the following connections must be made:

- SCLK (Serial Clock) to the SCLK pin of the Arduino
- CIPO (Controller In Peripheral Out) to the MISO pin of the Arduino
- COPI (Controller Out Peripheral In) to the MOSI pin of the Arduino
- CS (Chip Select) to a digital pin of the Arduino (for example pin 10)
- DC (Data/Command) to a digital pin of the Arduino (for example pin 9); data/command pin is used to distinguish if the message transmitted to the display is data or command
- RST (Reset) to a digital pin of the Arduino (for example pin 8)

Object initialization

To use the display functions in an Arduino sketch, it is necessary to import the library and its dependencies and instantiate an object of the class that manages the display, as in the following example:

```
1 #include <SPI.h>           // SPI protocol communication library
2 #include <Adafruit_GFX.h>   // core graphics library for displays
3 #include <Adafruit_HX8357.h> // library for the HX8357 display
4
5 #define TFT_CS 10  // chip select pin
6 #define TFT_DC 9   // data/Command pin
7 #define TFT_RST 8  // reset pin
8
9 // create an instance object of the display class
10 Adafruit_HX8357 tft = Adafruit_HX8357(TFT_CS, TFT_DC, TFT_RST);
```

Basic library functions

- `tft.begin()`: initializes the display and must be called in the `setup()` function before using any other function of the library
- `tft.setRotation(i)`: sets the rotation of the display, where *i* can be in range 0-3 and corresponds to a specific orientation of the display (0°, 90°, 180°, 270°)
- `tft.fillScreen(color)`: fills the entire display with the specified color
- `tft.setTextColor(color)`: sets the color for text rendering
- `tft.setTextSize(size)`: sets the size for text rendering, where *size* is an integer that scales the default text size (1 is default size, 2 is double size, etc.); default text size is 5x8 pixels.
- `tft.setCursor(x, y)`: sets the cursor position for text rendering at the specified coordinates
- `tft.print(text)`: prints the text at the current cursor position with current text, color and size
- `tft.println(text)`: same as `tft.print()`, plus moves the cursor to the beginning of next line
- `tft.drawPixel(x, y, color)`: draws a pixel at the specified coordinates with the specified color
- `tft.drawLine(x0, y0, x1, y1, color)`: draws a line based on the specified parameters
- `tft.drawRect(x, y, w, h, color)`: draws a rectangle based on the specified parameters
- `tft.fillRect(x, y, w, h, color)`: draws a filled rectangle based on the specified parameters
- `tft.drawCircle(x, y, r, color)`: draws a circle based on the specified parameters
- `tft.fillCircle(x, y, r, color)`: draws a filled circle based on the specified parameters

Pixel organization

The display is organized in a grid of pixels, where the top-left corner is the origin (0, 0) and the bottom-right corner is (479, 319). Every text character is rendered from the current cursor pixel-position to bottom-right corner of the display.

Color coding

The display is capable of 18-bit color depth, but the color is usually specified in 16-bit format (5 bits for red, 6 bits for green and 5 bits for blue) to save memory and bandwidth. Standard colors are pre-defined in the library as constants, for example:

- HX8357_BLACK: black color (0x0000)
- HX8357_RED: red color (0xF800)
- HX8357_GREEN: green color (0x07E0)
- HX8357_BLUE: blue color (0x001F)
- HX8357_CYAN: cyan color (0x07FF)
- HX8357_MAGENTA: magenta color (0xF81F)
- HX8357_YELLOW: yellow color (0xFFE0)
- HX8357_WHITE: white color (0xFFFF)

It is possible to create custom colors with the `tft.color565(r, g, b)` library function, where *r*, *g* and *b* are the red, green and blue components of the color in range 0-255.

Touchscreen functions

The display has a built-in resistive touchscreen that can be used to detect touch events and get the coordinates of the touch. Touchscreen is realized by two layers of conductive material separated by a small gap. When the screen is touched, the two layers come into contact and the resistance between them changes, allowing to get the coordinates and the pressure of the touch.

The LCD has 4 pins for the touchscreen (XP, XM, YP, YM) that can be connected to the Arduino (D7, A3, A2, D8 respectively) and managed with the following library functions:

- `Touchscreen(XP, YP, XM, YM, resistance)`: constructor of the touchscreen class that initializes the touchscreen object with the specified pins and the resistance value of the touchscreen
- `ts.getPoint()`: function that returns a structure with the coordinates (x, y, pressure) in range 0-4095 for the x and y coordinates and in range 0-255 for the pressure value; if the pressure value is greater than the `ts.pressureThreshold` (equal to 0), the touch is considered valid

The following code snippet shows how to get the coordinates of a touch event and map them to the display coordinates.

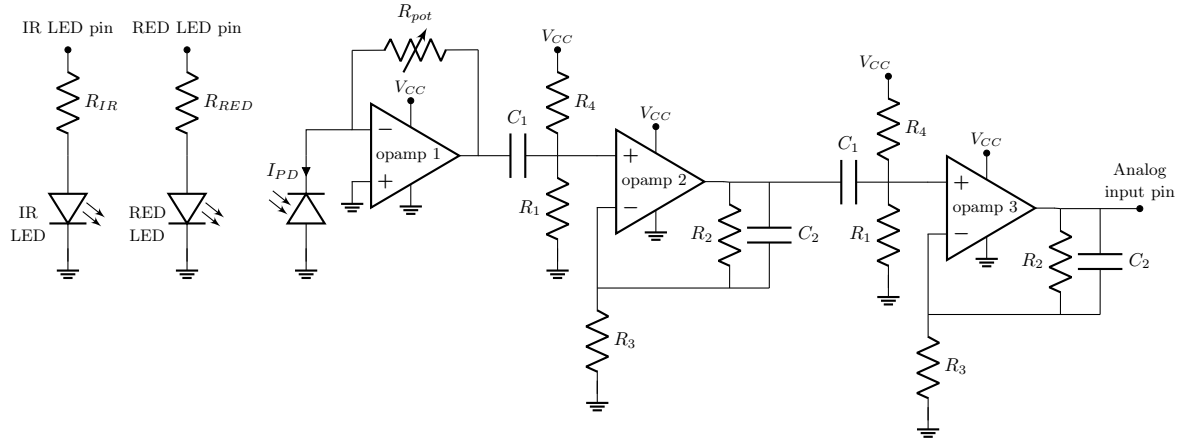
```
1 Touchscreen ts = Touchscreen(XP, YP, XM, YM, resistance); // object constructor
2 TSPoint p = ts.getPoint(); // get the coordinates of the touch (x, y, pressure)
3 p.x = map(p.x, 0, 4095, 0, tft.width()); // map x coordinate to display width
4 p.y = map(p.y, 0, 4095, 0, tft.height()); // map y coordinate to display height
5
6 // check if a touch event occurred by comparing the pressure value to the threshold
7 if (p.pressure > ts.pressureThreshold) {
8     // touch event has occurred at coordinates (p.x, p.y) with p.pressure
9 }
```

8.9 Heart rate monitor

Introduction to heart rate monitoring on Arduino

The idea for building a heart rate monitor is to detect the heart rate of a person by measuring the changes in reflectance of red and infrared light on the skin caused by the blood flow in the arteries. To do this an optical sensor is used in pair with a red LED and an infrared LED. The photodiode signal has a small pulsation component superimposed on a large DC component, so the signal must be conditioned and filtered to extract the pulsation component and get a clean signal that can be read by the ADC of the Arduino board.

Circuit components



Circuit is composed by:

1. two LEDs (red and infrared) connected in series with two resistors to limit the current through them, controlled by two digital pins of the Arduino board to turn them on and off
2. a photodiode connected in reverse bias to a transimpedance amplifier (TIA) (opamp 1) with a potentiometer in the feedback loop that converts the photodiode current into a voltage signal
3. a passive high pass filter with a cutoff frequency of $f_C \approx 0.8 \text{ Hz}$ and a voltage offset of 0.15 V (C_1 , R_1 and R_4) to remove the DC component of the signal and apply a small positive offset to the signal to make it suitable for op amps with single positive power supply
4. an active low pass filter with a cutoff frequency of $f_C \approx 3 \text{ Hz}$ and a gain of $A_v = 11$ (opamp 2, R_2 , R_3 and C_2) to filter the signal, remove high frequency noise and amplify the pulsation component of the signal
5. a second passive high pass filter with offset (C_1 , R_1 and R_4)
6. a second active low pass filter (opamp 3, R_2 , R_3 and C_2)
7. the output terminal connected to an analog input pin of the Arduino board

Oxygen saturation monitoring

Oxygen saturation can be estimated by measuring the ratio of the pulsation component of the red and infrared signals, since oxygenated and deoxygenated hemoglobin have different light absorption coefficients for red and infrared light. The ratio and the oxygen saturation is given by the following formulas:

$$R = \frac{\left(\frac{AC}{DC}\right)_{RED}}{\left(\frac{AC}{DC}\right)_{IR}} = \frac{(V_{\max,RED} - V_{\min,RED}) / V_{DC,RED}}{(V_{\max,IR} - V_{\min,IR}) / V_{DC,IR}} \approx \frac{(V_{\max,RED} - V_{\min,RED}) / V_{\min,RED}}{(V_{\max,IR} - V_{\min,IR}) / V_{\min,IR}}$$

$$\text{SpO}_2 = 97.94 + 1.15 \cdot R$$